



SKALA

생성형AI

sLLM 구현 및 Fine Tuning

2026.07

전임교수 : Byung-sun, Park

parkbs@sk.com

Ver.1.0

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

- 
- The background of the slide features a blurred image of hands typing on a laptop keyboard. Overlaid on this are several semi-transparent icons: a graduation cap, three interlocking gears, a globe, and a document with a checkmark. A large, faint watermark of a stylized 'A' is also visible across the center of the slide.
1. Why sLLM?
 2. sLLM 모델 탐색과 선택
 3. Inside the LLM : Transformer Recap
 4. How LLM Learns
 5. Fine-Tuning Map
 6. Fine-Tuning Techniques
 7. Fine-Tuning Troubleshooting

sLLM 구현 및 Fine Tuning

Why sLLM?

- 왜 작은 모델이 다시 주목받는가?
- 언제 sLLM을 선택하는가?
- sLLM의 주요 활용 영역

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

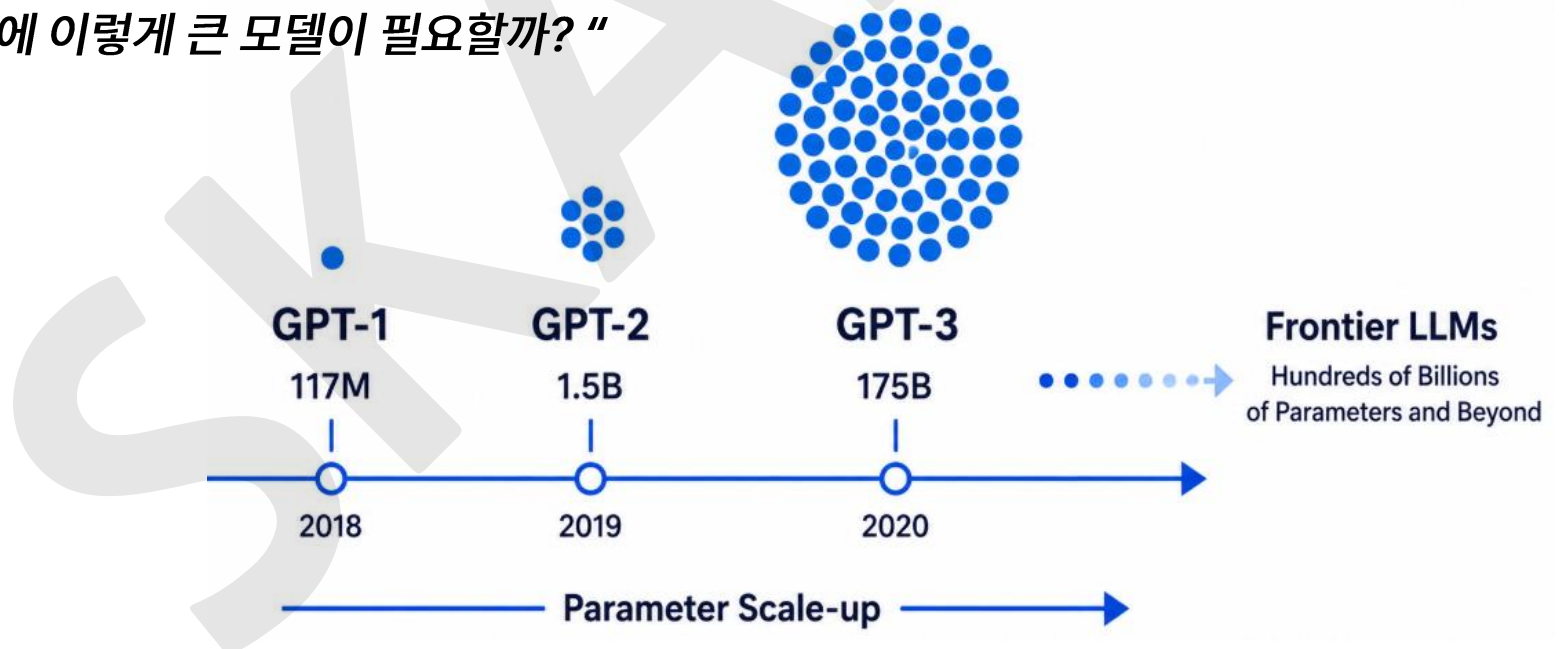
From Millions to Billions

LLM의 성능 경쟁은 더 많은 Parameter, Data, Compute를 투입하는 Scale-up을 중심으로 발전

Scale-up이 만든 LLM의 발전

- 더 많은 Parameters → 복잡한 패턴과 관계를 학습하고, 모델의 표현 능력을 확장
- 더 많은 Training Data → 다양한 지식과 문맥을 학습하며 범용성을 향상
- 더 많은 Compute → 대규모 모델과 데이터를 효과적으로 학습할 수 있는 기반 제공
- Scale-up은 동시에 더 많은 메모리, 학습·추론 비용, 컴퓨팅 인프라를 요구

“ 모든 AI 서비스에 이렇게 큰 모델이 필요할까? ”



왜 작은 모델이 다시 주목받는가?

큰 모델이 항상 최선은 아니다

대규모 LLM의 부담	작은 모델의 강점
높은 추론 비용	낮은 운영 비용
대용량 GPU 메모리 필요	제한된 자원에서도 구동
느린 응답과 네트워크 의존	빠른 응답과 로컬 실행
민감 데이터의 외부 전송	내부망·온디바이스 활용
범용 모델의 과도한 성능	특정 업무에 최적화 가능

- ✓ 대규모 LLM은 뛰어난 범용성을 제공하지만, 실제 서비스에서는 높은 비용과 운영 부담이 함께 발생
- ✓ 비용, 속도, 보안, 내부망 운영을 고려하면 작은 모델이 더 적합한 경우 관찰 → sLLM에 대한 기업의 실제 요구 증가
- ✓ 범용 성능 경쟁 → 비용·속도·보안 고려 → 업무에 맞는 작은 모델 선택도 하나의 옵션

“ 모든 문제에 작은 모델이 필요한 것은 아니다.

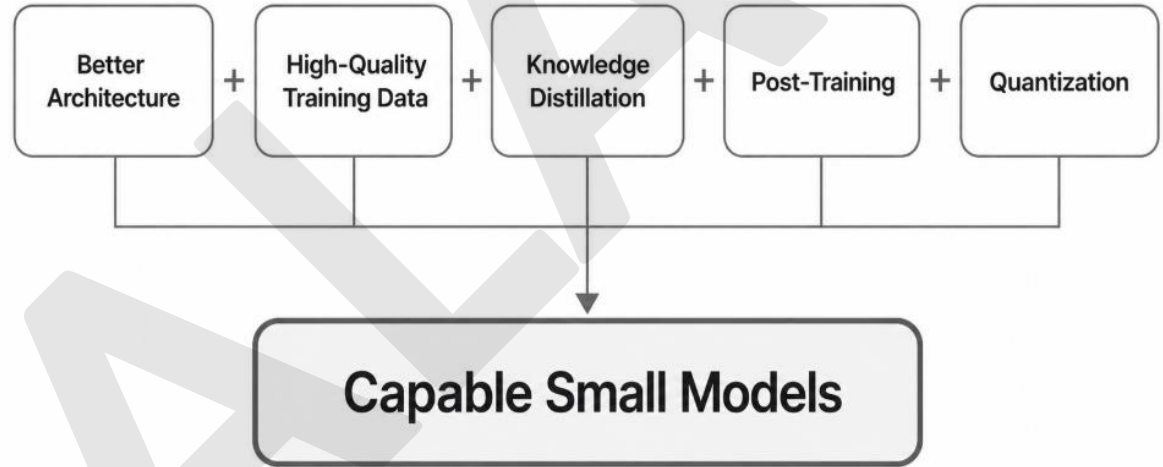
그러나 모든 문제에 가장 큰 모델이 필요한 것도 아니다. ”

왜 작은 모델이 다시 주목받는가?

과거의 작은 모델과 지금의 작은 모델은 다르다

작지만, 충분히 강해졌다

- 특정 업무에서는 대규모 모델에 근접한 성능
- 낮은 추론 비용과 빠른 응답 속도
- 제한된 GPU와 로컬 환경에서도 실행
- 보안이 중요한 내부망·온디바이스 배포
- 적은 자원으로 업무 맞춤형 미세조정



”

과거에는 작은 모델이 곧 제한된 성능을 의미했지만,

지금은 효율적인 설계와 학습을 통해 작은 모델의 성능 한계가 빠르게 확장

”

sLLM을 어떻게 정의할 것인가?

sLLM은 얼마나 작은 모델인가?

소형 언어 모델(Small Language Model, SLM or sLLM) 이란?

- sLLM은 LLM보다 상대적으로 적은 파라미터와 연산 자원을 사용하며, 제한된 환경이나 특정 목적에서도 효율적으로 활용할 수 있도록 설계된 언어 모델

판단 기준	상대적으로 작은 모델의 특성
파라미터 수	더 적음
필요한 연산량	더 적음
메모리 요구량	더 낮음
범용성	상대적으로 제한적
활용 범위	특정 과업과 환경에 집중

파라미터 기준

“ SLM의 분류 기준은 아직 명확하지 않다. 일부 연구는 10억 개 미만의 파라미터로 정의하지만, 다른 연구는 더 큰 모델과 비교한 상대적 개념으로 본다. 현재 통일된 정의에 대한 합의는 없다. ”

- 2025, ACM Computing Surveys, A Comprehensive Survey of Small Language Models in the Era of Large Language Models

언제 sLLM을 선택하는가?

상용 LLM을 사용할 수 없는 순간, 현실적 제약

”

하루 10여건의 고객 상담 Agent

- 고객의 다양한 상담에 대응 필요

- 사용자가 질문을 자유롭게 입력
- 다양한 주제에 대응해야 함
- 호출량이 적음
- 빠르게 서비스를 만들어야 함

→ 범용성·개발 속도

vs

”

하루 10만건의 고객문의 자동 분류 Agent

- 배송 문의 | 환불 요청 | 결제 오류 | 제품 불량 | 기타

- 입력과 출력 형식이 정해져 있음
- 동일한 작업을 대량으로 반복
- 빠른 응답과 낮은 건당 비용이 중요
- 고객 정보가 포함될 수 있음

→ 비용·속도·안정성

언제 sLLM을 선택하는가?

상용 LLM을 사용할 수 없는 순간, 현실적 제약

상황	사례	상용 LLM 사용의 문제
민감한 데이터를 다룰 때	환자 진료기록 요약, 고객 상담 분석	데이터를 외부 API로 전송하기 어려움
보안이 중요한 업무일 때	사내 기밀문서 검색, 국방·금융 시스템	외부 클라우드 연결 자체가 제한될 수 있음
인터넷을 사용할 수 없을 때	공장 설비, 자동차, 오프라인 단말기	네트워크 장애 시 서비스가 중단됨
호출량이 매우 많을 때	매일 수백만 건의 문서 분류	건당 비용은 작아도 전체 비용이 커짐
즉각적인 응답이 필요할 때	실시간 음성 명령, 이상 상황 감지	네트워크와 서버 처리시간이 누적됨
특정 업무만 반복할 때	스팸 분류, 품목 코드 분류	범용 LLM의 높은 성능이 과도할 수 있음
모델을 직접 통제해야 할 때	응답 형식 고정, 금지 표현 통제	모델 버전과 정책 변경의 영향을 받을 수 있음
조직에 맞게 조정해야 할 때	사내 용어·업무 규칙 적용	데이터와 모델을 직접 학습·관리하기 어려움

sLLM의 주요 활용 영역

sLLM Applications

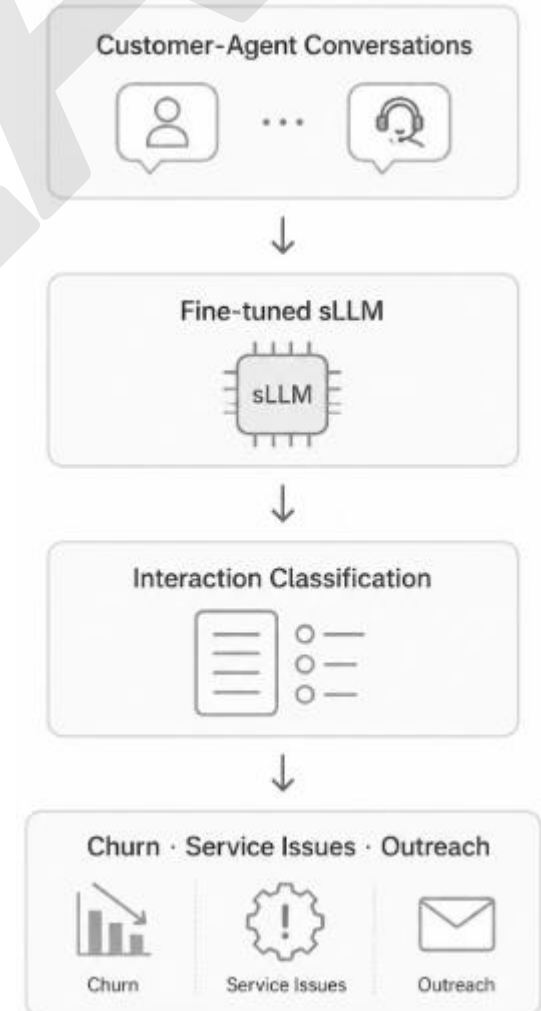
Enterprise AI : 반복되는 기업 업무 → 작고 빠른 전용 AI

- 기업에는 분류·추출·태깅·요약처럼 단순하지만 대량으로 반복되는 AI 업무가 존재
- 이러한 반복적이고 명확한 기업 업무에는 작고 효율적인 모델이 경쟁력이 될 수 있음

Case: AT&T | Customer Interaction Classification

- 미국 최대 통신사의 획기적인 고객 서비스 개선 사례
- 하루 수백만 건의 단순한 AI 업무에도 가장 큰 LLM이 필요할까?
- Problem
 - ✓ 수많은 고객 상담 내용을 지속적으로 분석하고 유형화해야 함
- Solution
 - ✓ 고객 상담 데이터를 Fine-tuned sLLM으로 자동 분류
 - ✓ 대규모 LLM 대신 특정 분류 업무에 최적화된 작은 모델 활용
 - ✓ 대량의 상담 데이터를 낮은 비용으로 빠르게 처리
- Results
 - ✓ 85% Cost Reduction
 - ✓ Performance & Quality Maintained

" Simple Task × High Volume → sLLM "



sLLM의 주요 활용 영역

sLLM Applications

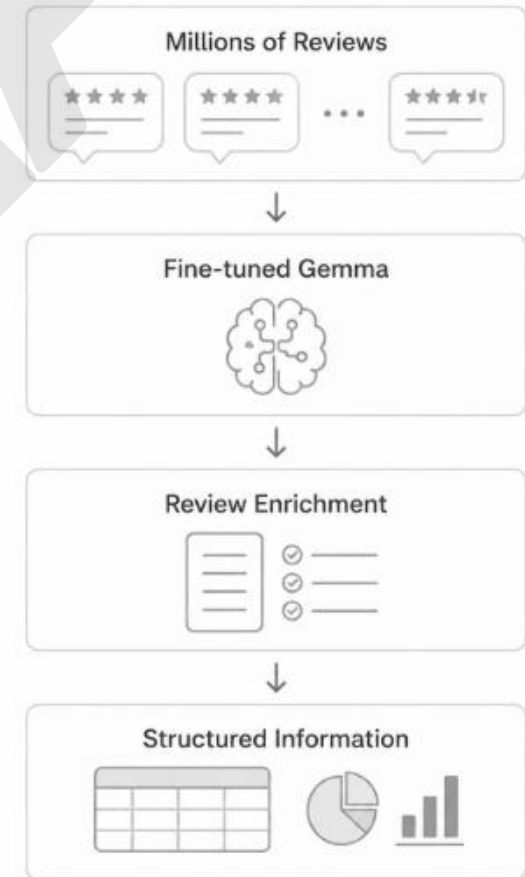
Enterprise AI : 반복되는 기업 업무 → 작고 빠른 전용 AI

- 기업에는 분류·추출·태깅·요약처럼 단순하지만 대량으로 반복되는 AI 업무가 존재
- 이러한 반복적이고 명확한 기업 업무에는 작고 효율적인 모델이 경쟁력이 될 수 있음

Case: Trustpilot | Review Analysis

- 덴마크 소재의 세계적인 온라인 Customer Review Platform
- 수억 건의 소비자 리뷰를 수집·분석하여 기업과 소비자의 의사결정을 지원
- 모든 리뷰를 거대 LLM으로 처리할 필요가 있는가?
- Problem
 - ✓ 월간 활성 사용자 6천만명, 하루 약 20만건의 신규 리뷰
 - ✓ 매일 대량의 짧은 텍스트가 계속 유입 + 단순하지만 반복되는 NLP Task
- Solution
 - ✓ 수백만 건을 처리해야 하기 때문에 매 리뷰마다 거대한 모델을 호출하는 것보다 작은 모델을 해당 업무에 최적화
 - ✓ Fine-tuned Gemma → Real-time architecture for big data
- Result
 - ✓ Real-time Processing · Scalability · Cost Efficiency

" High-volume + Latency → sLLM "



sLLM의 주요 활용 영역

sLLM Applications

Task-specific AI : 특정 Task → Fine-tuning으로 전문화

- General Capability를 조금 포기하더라도 특정 Task에서 충분한 성능 + 낮은 비용 + 빠른 속도를 얻기 위해서 sLLM을 Fine Tuning

Case: Epic System × Microsoft Phi-3

- 미국의 대표적인 EHR(Electronic Health Record) SW 기업
- *수많은 의료 기록에서 환자의 핵심 이력을 빠르게 파악할 수 있을까?*
- Problem
 - ✓ 복잡하고 방대한 Patient History → 의료진의 높은 Review 부담
- Solution
 - ✓ Microsoft Phi-3 (3.8B) 활용 검토 → Complex Patient History Summarization
 - ✓ Inpatient Insight, 응급실용 문제요약, 이식환자 임상노트 ..
- Why sLLM?
 - ✓ Lower Cost · Efficiency · Task-specific AI
- Results
 - ✓ 업무시간 30%+ 절감, 간호 기록 작성 85% 감소 등



sLLM의 주요 활용 영역

sLLM Applications

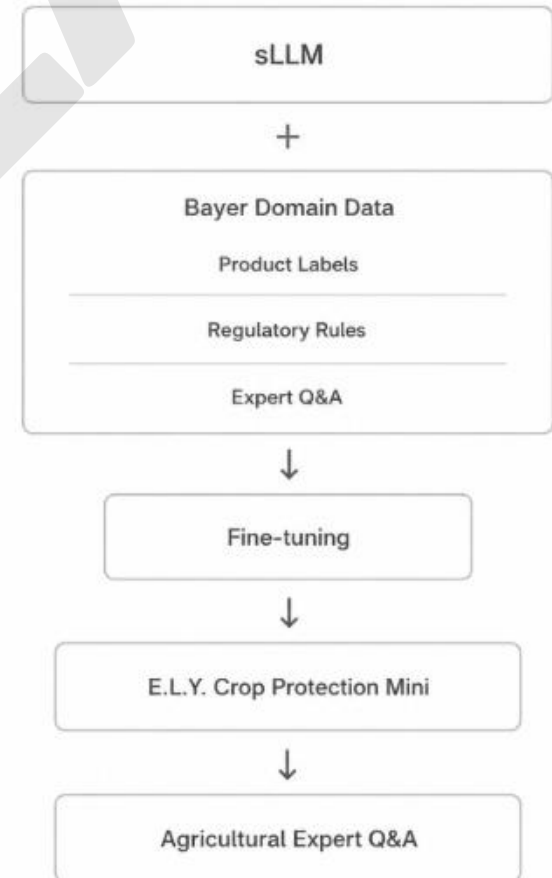
Task-specific AI : 특정 Task → Fine-tuning으로 전문화

- General Capability를 조금 포기하더라도 특정 Task에서 충분한 성능 + 낮은 비용 + 빠른 속도를 얻기 위해서 sLLM을 Fine Tuning

Case: Bayer | E.L.Y. Crop Protection Mini

- 아스피린으로도 유명한 독일의 글로벌 생명과학 기업, 제약/농업/작물과학
- *이 지역에서 이 작물에 이 제초제를 사용할 수 있는가?*
- Problem
 - ✓ 방대한 Crop Protection 제품·규제 정보
 - ✓ 지역·작물·시기별로 다른 적용 조건
 - ✓ 전문가의 문서 검색 및 기술팀 문의에 많은 시간 소요
- Solution
 - ✓ 농업 전문 Domain-specific sLLM
 - ✓ Bayer가 보유한 Product Label Data + Regulatory Rules + Expert-authored Q&A를 이용해 Fine-tuning
- Results
 - ✓ 며칠 또는 몇주가 걸리던 업무 → 30초 이내 단축
 - ✓ 5-10% Productivity Gain

"General-purpose Model → Domain-specific Model"



sLLM의 주요 활용 영역

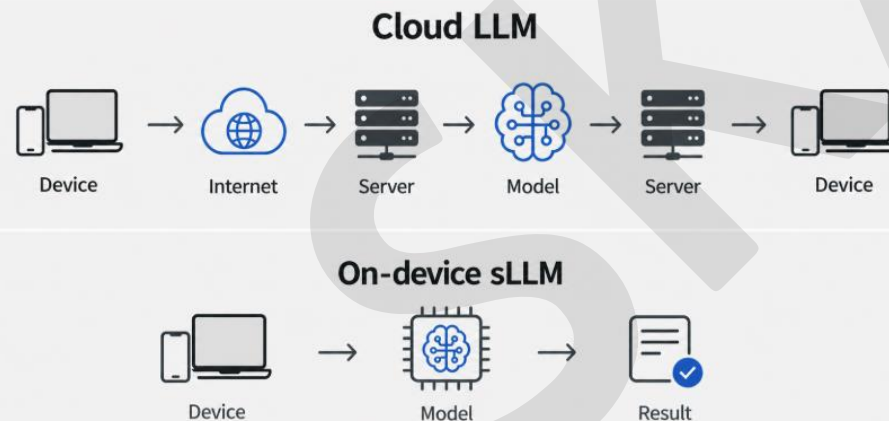
sLLM Applications

On-device AI : AI를 Cloud가 아니라 Device 안으로

- sLLM은 단순히 "작은 ChatGPT"가 아니라, AI를 서버에서 PC와 스마트폰 안으로 가져오는 핵심 기술
- Privacy, Offline, Latency focus

Case: Apple On-device Foundation Model

- Apple은 Apple Intelligence를 위해 약 3B parameter의 On-device Foundation Model을 개발
- 이를 글 다듬기, 알림 요약, 텍스트 이해 등의 기능에 활용



Writing Benchmarks

Server

Summarization



On-Device

Summarization



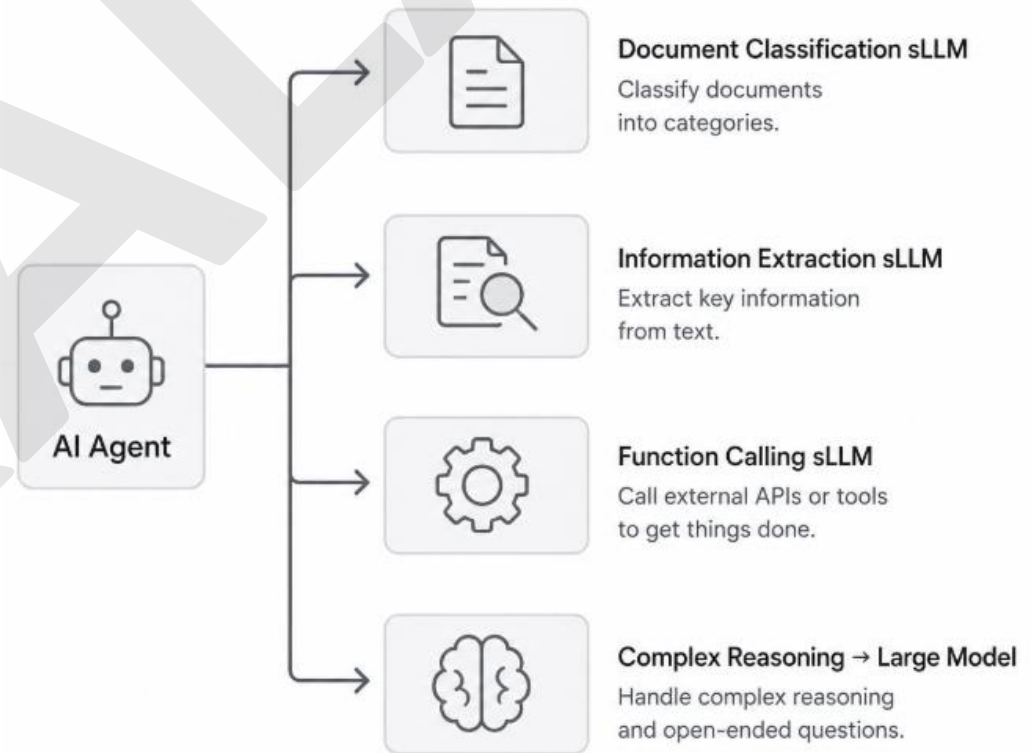
sLLM의 주요 활용 영역

sLLM Applications

AI Agent : 하나의 거대한 AI → 여러 전문 AI의 조합

- 최근 AI 시스템에서는 작은 전문 모델과 큰 범용 모델을 조합하는 구조가 중요해지고 있음
- Agent / Router는 요청에 따라 적절한 모델과 도구를 선택하고, 여러 전문 AI를 하나의 시스템으로 연결
- 향후 AI 엔지니어에게 요구되는 역량 중 하나는 업무 특성에 따라 적절한 모델을 선택하고 조합하는 역량
- Efficiency : 단순한 작업은 작은 모델이 처리 → GPU 자원과 비용 절감
- Specialization : 각 sLLM을 특정 데이터로 Fine-tuning → 특정 업무에 최적화
- Performance : 어려운 문제만 Large LLM으로 전달 → 필요한 곳에 큰 모델의 추론 능력 사용

" One Model for Everything → Right Model for Each Task "



sLLM 구현 및 Fine Tuning

sLLM 모델 탐색과 선택

- sLLM Model Families
- sLLM 모델 살펴보기
- Model Selection : Practical Guide

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

sLLM Model Families

목적과 자원에 맞는 모델을 선택

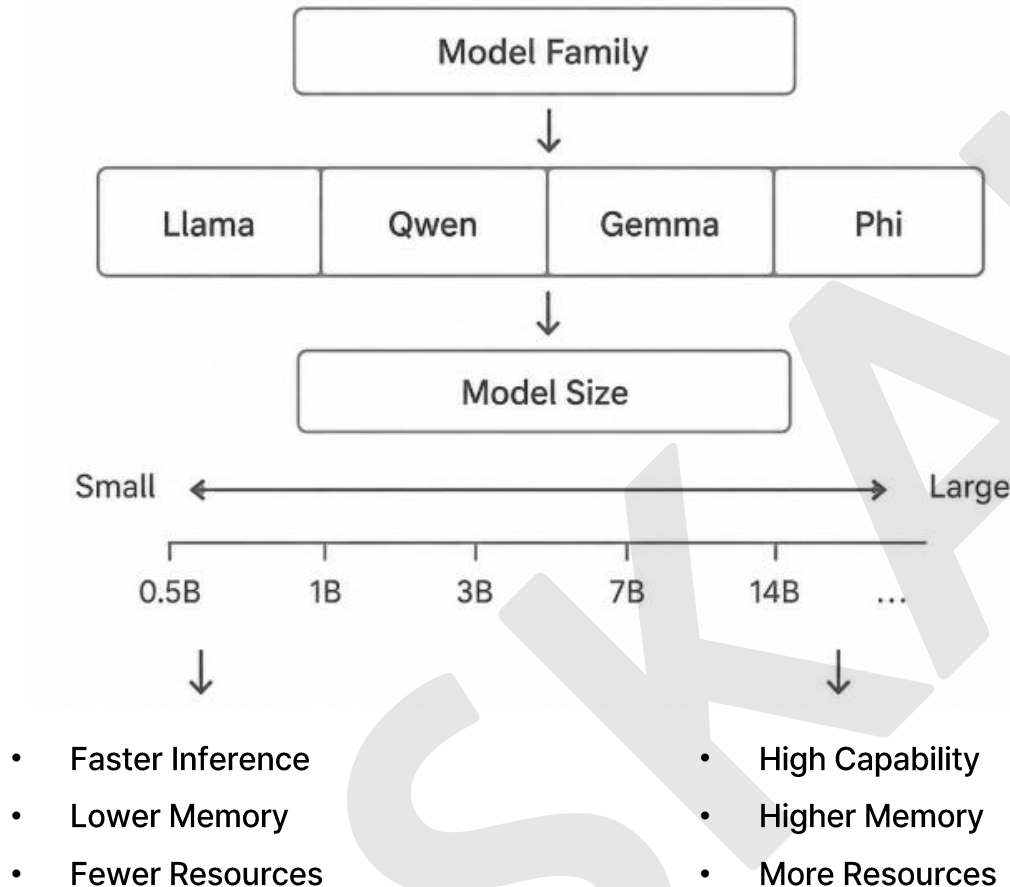
➤ 대표적인 Open-weight Model Family

Model Family	개발사	주요 특징	적합한 활용
Llama	Meta	Open-weight 생태계를 확산시킨 대표 모델군, 다양한 도구·커뮤니티 지원	범용 LLM, Fine-tuning, 연구·서비스
Qwen	Alibaba	소형부터 대형까지 폭넓은 규모, 다국어·코딩·추론 등 다양한 모델군	범용 AI, Coding, Agent, Fine-tuning
Gemma	Google	Gemini 기술을 기반으로 한 경량 Open Model, On-device부터 Server까지 다양한 크기 제공	Local/Edge AI, Multimodal, Fine-tuning
Phi	Microsoft	적은 Parameter로 높은 효율을 지향하는 Small Language Model 계열	Task-specific AI, Edge, 기업 업무 특화

✓ 하나의 모델이 있는 것이 아니라, 하나의 Model Family 안에 여러 크기와 용도의 모델이 존재

sLLM Model Families

목적과 자원에 맞는 모델을 선택



- ✓ Model Family를 선택하고, Task와 Resource에 맞는 Model Size를 선택
- ✓ 고려요소 : Task · Quality · Memory · Latency · Cost
- ✓ 어떤 모델을 Fine-tuning할 것인가?

sLLM의 Capability

- 모델의 종합적인 문제 해결 능력, 더 다양하고 복잡한 Task를 수행할 수 있는 잠재적 능력
- Reasoning, Language Understanding, Coding..
- 복합적인 이해 + 추론 + 생성이 필요한 업무는 일반적으로 더 높은 capability가 요구
- Model Size가 커질수록 Capability가 높아지는 것은 일반적인 경향이지 절대적인 법칙은 아님

“ Model Selection = Resource Efficiency ↔ Capability ”

sLLM Model Families

목적과 자원에 맞는 모델을 선택

The screenshot shows the Hugging Face website interface. At the top, there's a navigation bar with 'Hugging Face' logo, a search bar, and links for Models, Datasets, Spaces, Buckets, Docs, and Pricing. Below the navigation bar is a yellow banner with a message about joining an organization. The main content area is divided into two columns. The left column contains filters for Tasks (Text Generation, Image-Text-to-Text, etc.), Parameters (a slider from < 1B to > 500B), Libraries (PyTorch, TensorFlow, etc.), and Apps (vLLM, llama.cpp, etc.). The right column displays a list of models under the 'Models' tab, filtered by 'qwen'. The models listed include DavidAU/Qwen3.6-27B-Fable-Fusion-711-Uncensored-Heretic-NM-DAU-NEO-MAX-MTP-GGUF, ethanfel/Qwen3-VL-32B-Ultra-Heretic-H3-ComfyUI-INT8-ConvRot, sakamakismile/Qwen3-VL-32B-Heretic-MiniMax-H3-NVFP4, LuffyTheFox/Qwen3.6-35B-A3B-Uncensored-Genesis-Hermes-V7-GGUF, HauhauCS/Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggressive, DavidAU/Qwen3.5-9B-The-Defiant-Fable-Uncensored-Heretic-NEO-IMATRIX-MAX-MTP-GGUF, and Qwen/Qwen3.6-27B. Each model entry shows its name, task type, size, update time, and download/like counts.

Hugging Face

Search models, datasets, users...

Models Datasets Spaces Buckets NEW Docs Pricing

Hugging Face is way more fun with friends and colleagues! Join an organization

Dismiss this message

Main Tasks Libraries Languages Licenses Other

Tasks

Text Generation Any-to-Any

Image-Text-to-Text Image-to-Text

Image-to-Image Text-to-Image

Text-to-Video Text-to-Speech + 44

Parameters

< 1B 6B 12B 32B 128B > 500B

Libraries

PyTorch TensorFlow JAX

Transformers Diffusers GGUF + 47

Apps

vLLM llama.cpp MLX LM LM Studio

Ollama Jan Draw Things + 10

Models 296,688 qwen Base only Inference Available Sort: Trending

DavidAU/Qwen3.6-27B-Fable-Fusion-711-Uncensored-Heretic-NM-DAU-NEO-MAX-MTP-GGUF

Image-Text-to-Text · 27B · Updated about 21 hours ago · 2.52M · 1.9k

ethanfel/Qwen3-VL-32B-Ultra-Heretic-H3-ComfyUI-INT8-ConvRot

Image-Text-to-Text · Updated 6 days ago · 458

sakamakismile/Qwen3-VL-32B-Heretic-MiniMax-H3-NVFP4

Updated 8 days ago · 161

LuffyTheFox/Qwen3.6-35B-A3B-Uncensored-Genesis-Hermes-V7-GGUF

Image-Text-to-Text · 35B · Updated 3 days ago · 436k · 472

HauhauCS/Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggressive

Image-Text-to-Text · 35B · Updated Apr 17 · 2.01M · 3.39k

DavidAU/Qwen3.5-9B-The-Defiant-Fable-Uncensored-Heretic-NEO-IMATRIX-MAX-MTP-GGUF

Image-Text-to-Text · 9B · Updated 5 days ago · 419k · 344

Qwen/Qwen3.6-27B

Image-Text-to-Text · 28B · Updated Apr 24 · 6.58M · 2.23k

sLLM Model Families

목적과 자원에 맞는 모델을 선택

 **Hugging Face**

[Models](#) [Datasets](#) [Spaces](#) [Buckets](#) NEW

Hugging Face is way more fun with friends and colleagues!  [Join an organization](#)

Qwen's Collections

Qwen-AgentWorld

Qwen-Image-Bench

WebWorld

Qwen-Scope

Qwen3.6

Qwen3.5

Qwen3-Coder-Next

Qwen3-ASR

Qwen3-TTS

Qwen3-VL-Reranker

Qwen3.5

updated Mar 9

 Qwen/Qwen3.5-397B-A17B

 Image-Text-to-Text • 403B • Updated Apr 24 • 316k • ⚡ • ❤️ 1.55k

 Qwen/Qwen3.5-397B-A17B-FP8

 Image-Text-to-Text • 403B • Updated Apr 24 • 836k • ❤️ 182

 Qwen/Qwen3.5-122B-A10B

 Image-Text-to-Text • 125B • Updated Apr 24 • 2.14M • ⚡ • ❤️ 605

 Qwen/Qwen3.5-122B-A10B-FP8

 Image-Text-to-Text • 125B • Updated Apr 24 • 1.34M • ❤️ 112

Base Model vs Instruct Model

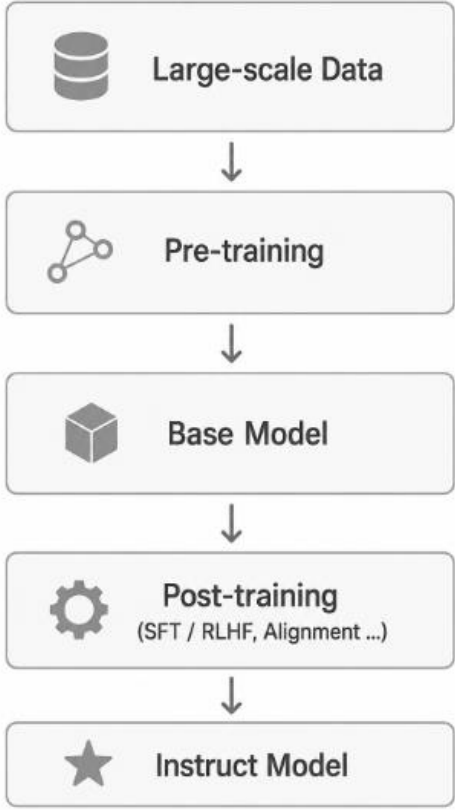
Model Family에서 목적과 자원에 맞는 모델을 선택

 Qwen/Qwen3.5-4B
Image-Text-to-Text • 5B • Updated Mar 2 • 6.65M • 805

 Qwen/Qwen3.5-4B-Base
Image-Text-to-Text • 5B • Updated Apr 23 • 220k • 80

	Instruct Model	Base Model
상태	Post-trained 모델	Pre-trained Base 모델
대화/지시 수행	바로 가능	아직 미흡
질문 → 답변	지시,수행,대화에 최적화	다음 토큰 예측이 기본 목적
바로 Chat 활용	적합	부적합
추가 Fine-tuning	가능	가능
주요 용도	Chat, Agent, 추가 SFT	계속된 Pretraining, 전문적인 FT

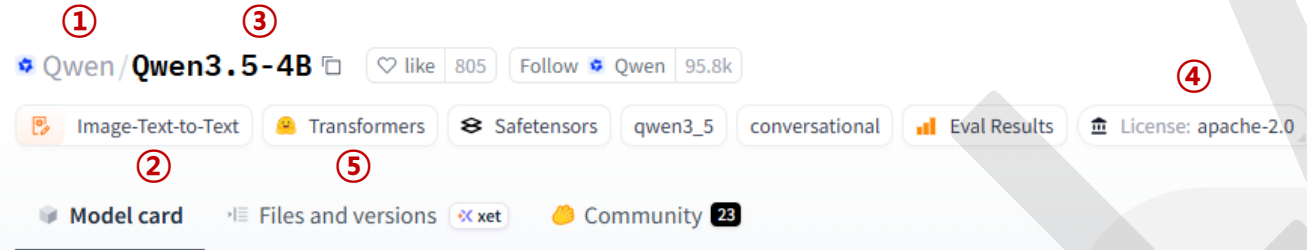
[Model Workflow]



sLLM 모델 살펴보기

Hugging Face Model Card 읽는 법

✓ 최신 모델보다 중요한 것은 “내 Task와 환경에 적합한 모델인가?”



Qwen3.5-4B



♥ Qwen Chat

This repository contains model weights and configuration files for the post-trained model in the Hugging Face Transformers format.

These artifacts are compatible with Hugging Face Transformers, vLLM, SGLang,

Model Card에서 확인해야 할 핵심 정보

(1) Publisher

Official or Community?

(2) Task

Text / Vision / Multimodal?

(3) Size & Type

1B / 4B / 8B · Base / Instruct

(4) License

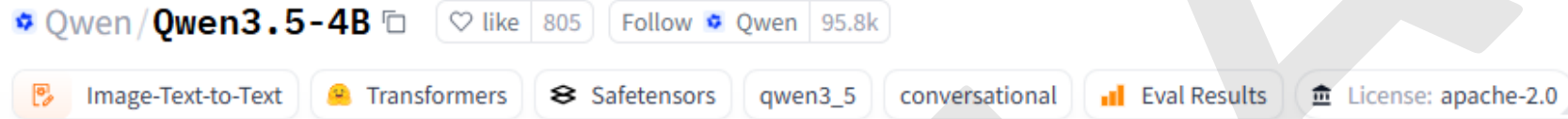
Commercial use allowed?

(5) Compatibility

Transformers / PEFT / Quantization

sLLM 모델 살펴보기

Hugging Face Model Card 읽는 법



표시	의미	실무적으로 보는 것
Qwen / Qwen3.5-4B	Publisher / Model	원 개발사가 공개한 모델인가? 파생모델인가?
Image-Text-to-Text	Task	입력/출력 및 모델 용도, 무엇을 하는 모델인가?
Transformers	Library	어떤 라이브러리에서 사용할 수 있는가?
Safetensors	Weight Format	모델 weight 저장 형식
qwen_3_5	Architecture/Model tag	모델 계열·구조
conversational	Capability/Task tag	대화형 사용을 고려한 모델
Eval Results	Evaluation	공개된 평가 결과는 존재하는가?
apache-2.0	License	상업적 활용에 문제가 없는가? 사용·수정·배포 조건

sLLM 모델 살펴보기

Hugging Face Model Card 읽는 법

Model tree for Qwen/Qwen3.5-4B

Base model	Qwen/Qwen3.5-4B-Base
Finetuned (101)	this model
Adapters	485 models
Finetunes	464 models
Merges	10 models
Quantizations	343 models

- Qwen3.5-4B-Base → Post-training → Qwen3.5-4B
- Base Model을 기반으로 fine-tuned model이 101개
- Adapters : LoRA 등 PEFT로 학습한 추가 가중치를 Adapter 형태로 배포
- Finetunes : 원 모델을 특정업무, 전문분야 등 추가 학습하여 별도의 파생 모델로 배포
- Merges : 여러 모델이나 학습 결과의 weight를 결합하여 만든 모델
- Quantization : 저장밀도 형태로 변환하여 메모리와 추론 자원 요구량을 낮춘 모델

License도 모델 선택 기준이다

Open-weight ≠ Free to use without restrictions

✔ Permissive Open Source (사용 제약이 적은 오픈소스)

- 상업적 이용, 수정, 배포를 폭넓게 허용하며, 소스/모델을 수정해 만든 결과물도 반드시 동일한 오픈소스 라이선스로 공개할 필요가 없음

License	특징	핵심 차이
MIT	매우 단순하고 자유로운 라이선스	저작권·라이선스 고지 유지가 핵심
BSD	MIT와 유사한 permissive license	버전에 따라 저작자·기관 이름을 홍보에 사용하는 것을 제한
Apache-2.0	자유로운 사용 + 보다 상세한 법적 조항	명시적인 특허(Patent) 사용권과 NOTICE 등의 규정 포함

- 기업 입장에서 Apache-2.0의 중요한 특징 중 하나는 명시적인 patent grant와 patent termination 조항을 가지고 있는 점

Apache-2.0

Commercial Use ✓

Modification / Fine-tuning ✓

Redistribution ✓

Explicit Patent Grant ✓

License도 모델 선택 기준이다

Open-weight ≠ Free to use without restrictions

✔ Model-specific License

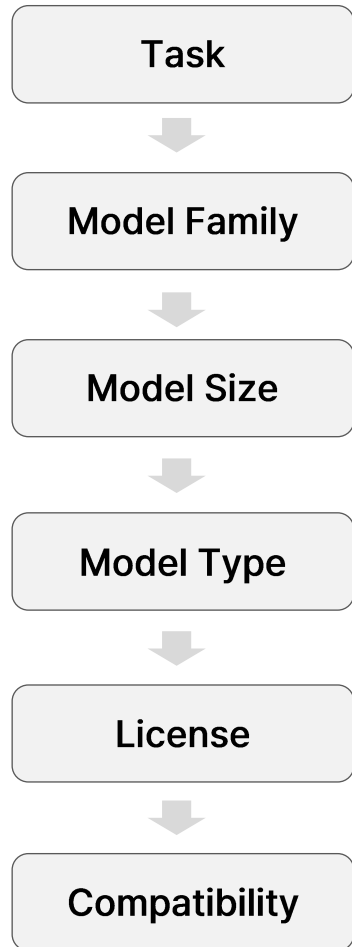
- 모델 개발사가 해당 모델을 위해 별도로 정한 사용 조건
- 상업적 이용·수정·Fine-tuning이 가능하더라도 사용 제한, 재배포 조건 등 모델별 약관을 별도로 확인해야 함
- Ex) Gemma Series = 상업적 활용 가능 + Google의 사용 제한 정책 준수 필요
- Ex) Llama = 상업적 활용 가능 + Meta의 모델별 Community License 조건 확인

✔ Restricted License

- Research Only / Non-commercial / Academic Only 등
- 연구·교육 등 특정 목적으로 사용 범위를 제한하는 라이선스
- 모델이 공개되어 있어도 상업적 이용·서비스 배포·파생 모델 활용 등이 제한될 수 있음
- Ex) BrainIAC, '26년 공개 뇌 MRI foundation model : Research-Only License

Model Selection : Practical Guide

어떤 sLLM을 선택해야 할까?



무엇을 시킬 것인가?

- Text Generation · Classification · Coding · Multimodal ...

어떤 계열을 사용할 것인가?

- Qwen · Llama · Gemma · Phi ...

내 환경에서 실행 가능한가?

- 1B · 4B · 8B · 14B ...
- Capability ↔ Memory · Latency · Cost

어떻게 사용할 것인가?

- 일반적인 SFT / QLoRA → Instruct 우선 검토
- Continued Pre-training → Base 검토

실제 서비스에 사용할 수 있는가?

- Commercial Use / Modification / Redistribution 확인
- Apache-2.0 · MIT · Model-specific · Restricted ...

내 환경에서 실제로 Fine-tuning 가능한가?

- GPU / Quantization / Framework 지원 확인
- Transformers · PEFT · TRL · bitsandbytes · Unsloth ...

모델에 부족한 것을 어떻게 채울 것인가?

Prompt vs RAG vs Fine-Tuning

모델에 부족한 것	우선 고려	역할
지시·맥락 제공	Prompt	입력에 필요한 지시와 Context 제공
외부·최신 지식 활용	RAG	필요한 지식을 검색하여 Context로 제공
특정 Task·행동 학습	Fine-Tuning	모델의 행동과 응답 패턴을 학습

”

올해 우리 회사의 출장비 규정이 얼마인가?

- 사내 규정 Q&A 시스템

→ RAG : 규정 문서를 찾아 응답

”

고객 문의에 항상 우리 회사 상담 양식으로 답변해줘

- 고객 상담 자동화 시스템

→ Fine-Tuning : 원하는 응답 방식과 패턴을 학습

- ✓ Knowledge → RAG / Behavior → Fine-Tuning
- ✓ Fine-Tuning으로도 지식을 넣을 수 있지만, 변하는 외부 지식을 활용하는 문제라면 우선 RAG를 검토하는 경향
- ✓ Fine-Tuning의 주목적은 지식을 저장하는 것이 아니라, 모델의 행동을 바꾸는 것

모델에 부족한 것을 어떻게 채울 것인가?

Prompt vs RAG vs Fine-Tuning

Q. 도메인 특화 sLLM은 RAG가 더 적합하지 않아?

예를 들어 금융 전문 sLLM을 구축하는 경우

“2026년 현재 회사의 대출상품 금리는 얼마인가?” 유형의 질의

→ RAG가 적합

→ 금리가 바뀔 때마다 모델을 다시 Fine-Tuning할 이유가 없음

“이 고객의 상담 기록을 보고 정상 / 관심 / 이탈위험으로 분류하라.” 유형의 질의

→ Fine-Tuning이 적합

→ 반복되는 입력/출력 패턴 자체를 모델이 안정적으로 수행하도록 만드는 문제이기 때문

모델에 부족한 것을 어떻게 채울 것인가?

Prompt vs RAG vs Fine-Tuning

Q. *Fine-Tuning*의 주목적이 지식의 저장이지 아니라 모델의 행동을 바꾸는 것이라면 프롬프트와 무엇이 다를까?

예를 들어 금융 전문 sLLM을 구축하는 경우

“이 고객의 상담 기록을 보고 정상 / 관심 / 이탈위험으로 분류하라.” 유형의 질의

Case1)

긴 System Prompt + 5개 Example + Input → Qwen 1.7B → 결과

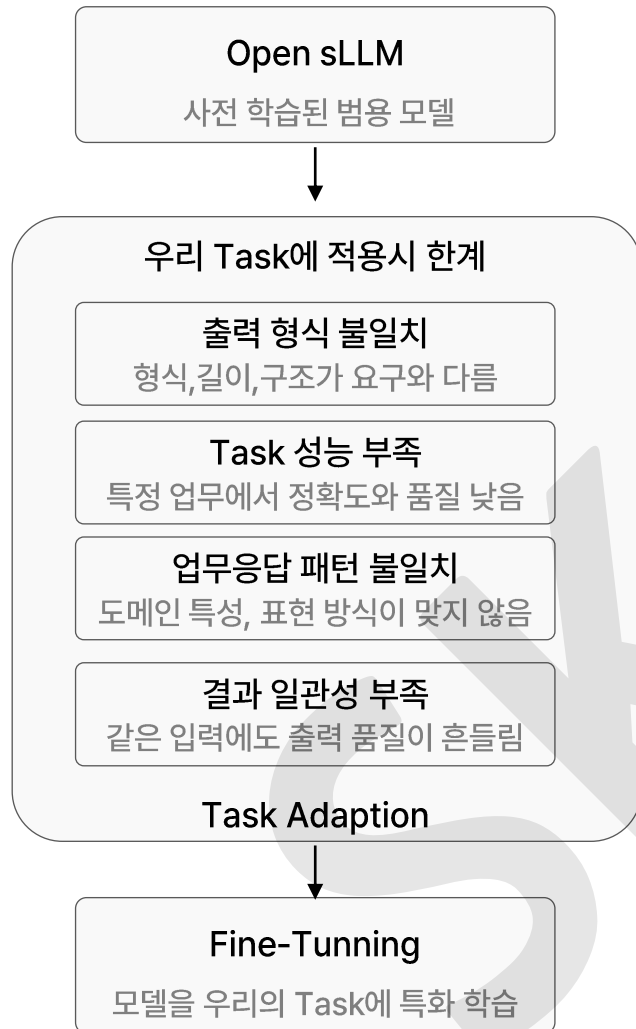
Case2)

Input → Specialized Qwen 1.7B → 원하는 결과

- ✓ *Fine-Tuning*의 가치는 단순히 “도메인 지식을 넣는다”가 아니라,
작은 모델도 특정 업무를 안정적이고 반복적으로 수행하도록 전문화 하는데 있음

From sLLM to Fine-Tuning

Fine-Tuning은 범용 sLLM을 특정 Task에 특화된 AI로 바꾸는 과정



- ✓ 좋은 범용 모델이 곧 좋은 업무 모델은 아님
- ✓ 범용 sLLM을 우리의 Task와 응답 방식에 맞게 Adaptation 필요
→ Fine-Tuning
- ✓ 그런데 LLM의 Fine-Tuning에서는 무엇을 정답으로 놓고 학습하는 것일까?
- ✓ 수십억 개의 Parameter를 전부 다시 학습해야 할까?

sLLM 구현 및 Fine Tuning

Inside the LLM : Transformer Recap

- Math Essentials for Understanding Transformers
- Scaled Dot-Product Attention : Self Attention
- Masked Self Attention : Causal Masking

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Dot Product

내적의 의미

Calculation

- 두 벡터 $\mathbf{a}$, $\mathbf{b}$ 가 이루는 각이 θ 일 때 다음을 두 벡터 $\mathbf{a}$, $\mathbf{b}$ 의 내적이라 함

$$\vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos \theta = \sum_{i=1}^d a_i b_i$$

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\mathbf{a}^T \mathbf{b} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

$$= 1 \times 4 + 2 \times 5 + 3 \times 6$$

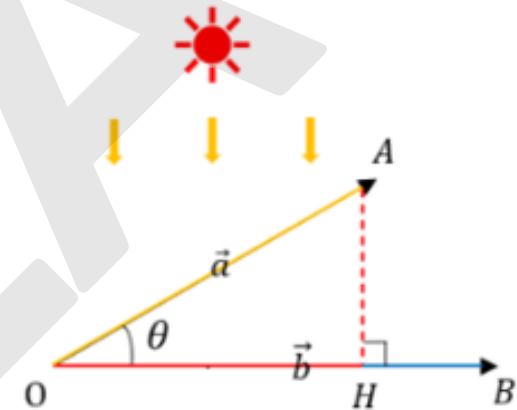
$$= 4 + 10 + 18 = 32$$

Dot Product

내적의 의미

Implicit meaning

- 벡터의 내적은 기하학적으로 한 벡터를 다른 벡터 위로 정사영시킨 길이와 그 다른 벡터의 길이를 곱한 것을 의미
- 크기와 방향을 모두 고려한 유사도 개념
- 정사영된 길이가 짧다는 것 = 벡터간 각도가 크다 = 유사도가 작다
- 정사영된 길이가 길다는 것 = 벡터간 각도가 작다 = 유사도가 크다



$$\begin{aligned}\vec{a} \cdot \vec{b} &= |\vec{a}| \times |\vec{b}| \times \cos \theta \\ &= |\vec{b}| \times |\vec{a}| \times \cos \theta \\ &= \overline{OH} \times \overline{OB}\end{aligned}$$

Matrix Multiplication

$$\begin{array}{c} A(2 \times 3) \\ \begin{bmatrix} 2 & 1 & 3 \\ 4 & 0 & 2 \end{bmatrix} \end{array} \times \begin{array}{c} B(3 \times 2) \\ \begin{bmatrix} 5 & 2 \\ 1 & 4 \\ 3 & 7 \end{bmatrix} \end{array} = \begin{array}{c} AB(2 \times 2) \\ \begin{bmatrix} 20 & 29 \\ 26 & 22 \end{bmatrix} \end{array}$$

$$(AB)_{11} = (2 \cdot 5) + (1 \cdot 1) + (3 \cdot 3) = 20$$

$$\begin{array}{c} A \\ \begin{bmatrix} 2 & 1 & 3 \\ 4 & 0 & 2 \end{bmatrix} \end{array} \times \begin{array}{c} B \\ \begin{bmatrix} 5 & 2 \\ 1 & 4 \\ 3 & 7 \end{bmatrix} \end{array} = \begin{array}{c} AB \\ \begin{bmatrix} 20 & 29 \\ 26 & 22 \end{bmatrix} \end{array}$$

$$(AB)_{12} = (2 \cdot 2) + (1 \cdot 4) + (3 \cdot 7) = 29$$

✓ 행렬곱(Matrix Multiplication)의 각 원소는 행(Row) 벡터와 열(Column) 벡터의 내적(Dot Product)

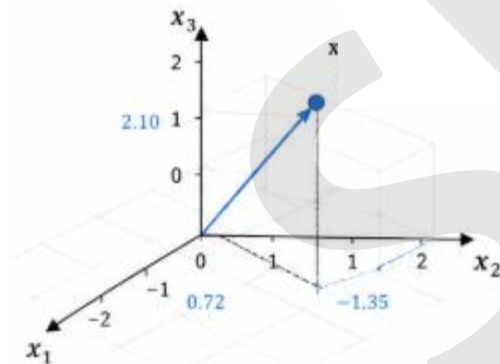
✓ Matrix Multiplication = Row × Column Dot Products

선형변환, linear transformation

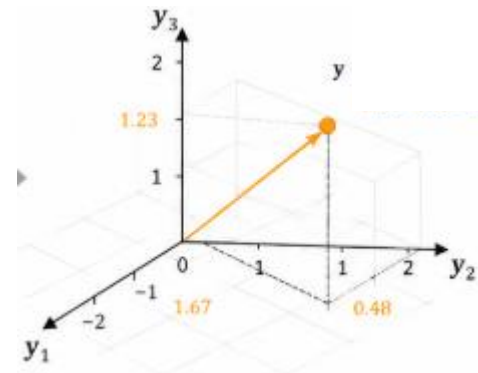
$$\mathbf{x} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}_{(1 \times 3)} \quad \mathbf{W} = \begin{bmatrix} 2 & 0 \\ 1 & -1 \\ 0 & 1 \end{bmatrix}_{(3 \times 2)} = \mathbf{y} = \begin{bmatrix} 4 & 1 \end{bmatrix}_{(1 \times 2)}$$

- 3차원 벡터 $\mathbf{x}$ 는 행렬 $\mathbf{W}$ 와 곱을 통해 2차원 벡터로 선형변환 됨

$$\mathbf{x} = \begin{bmatrix} 0.72 & -1.35 & 2.10 \end{bmatrix}_{(1 \times 3)} \quad \mathbf{W} = \begin{bmatrix} 0.80 & -0.40 & 0.30 \\ -0.50 & 0.90 & -0.10 \\ 0.30 & 0.20 & 0.70 \end{bmatrix}_{(3 \times 3)} = \mathbf{y} = \begin{bmatrix} 1.67 & 0.48 & 1.23 \end{bmatrix}_{(1 \times 3)}$$



- 선형변환은 벡터 $\mathbf{x}$ 를 새로운 좌표로 이동시킴



Definition

- 선형변환은 벡터의 덧셈과 배수 관계를 유지하면서, 벡터를 다른 벡터로 매핑하는 함수, 행렬곱은 이러한 선형변환을 계산하는 방법
- 벡터를 회전·확대·축소·반사하거나 다른 차원의 벡터로 바꾸되, 공간의 직선성과 원점을 보존하는 변환

Softmax function

Softmax: Scores to Probabilities

Concept

- Softmax는 여러 실수 점수를 비교 가능한 확률 분포로 변환
- 각 점수가 전체에서 차지하는 상대적 비중을 계산
- 모든 출력값은 0과 1 사이
- 출력값의 합은 1
- 입력값의 상대적인 크기 관계를 유지

Score

2.1 

1.3 

0.2 

-0.5

Softmax

0.58 

0.26 

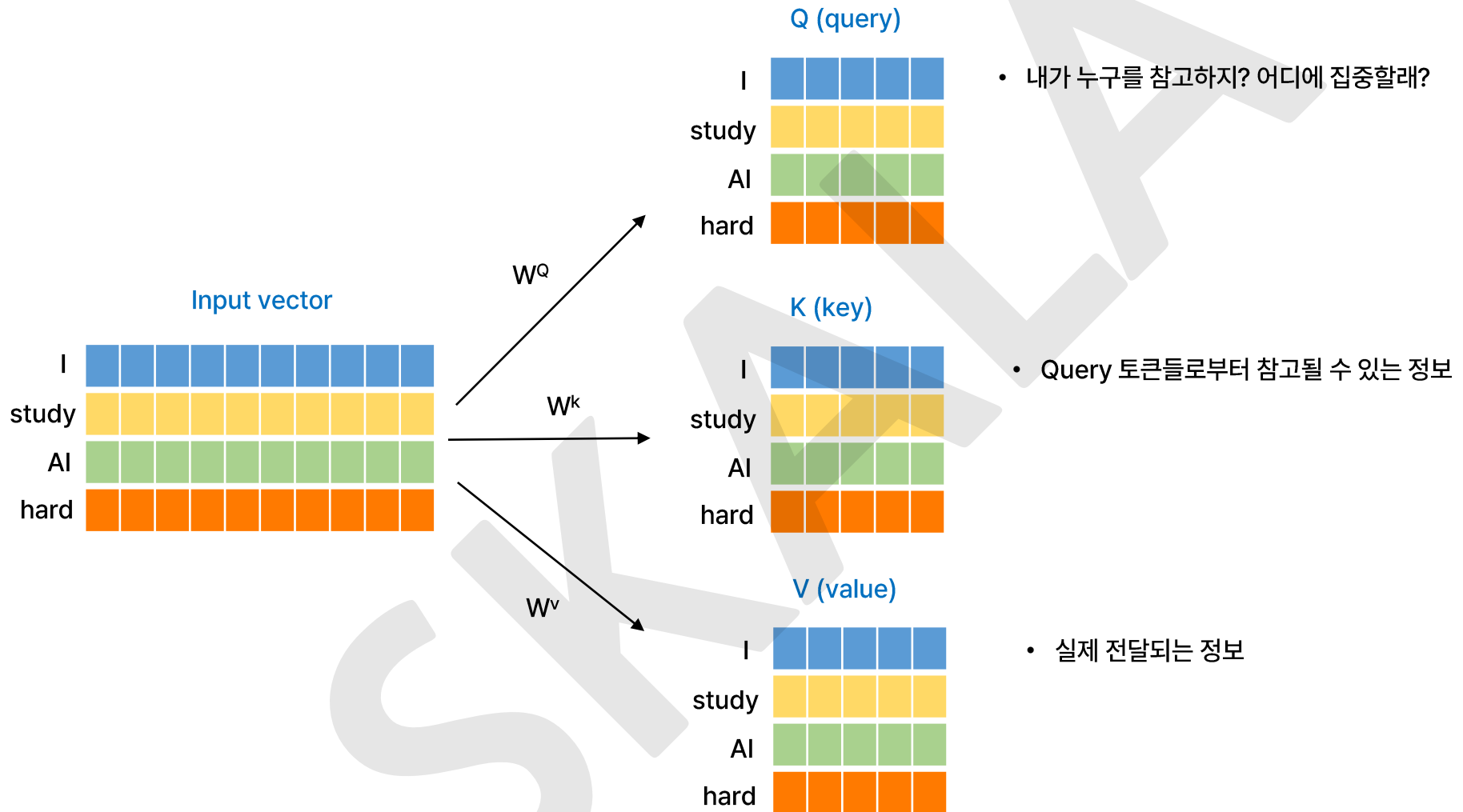
0.09 

0.07 

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

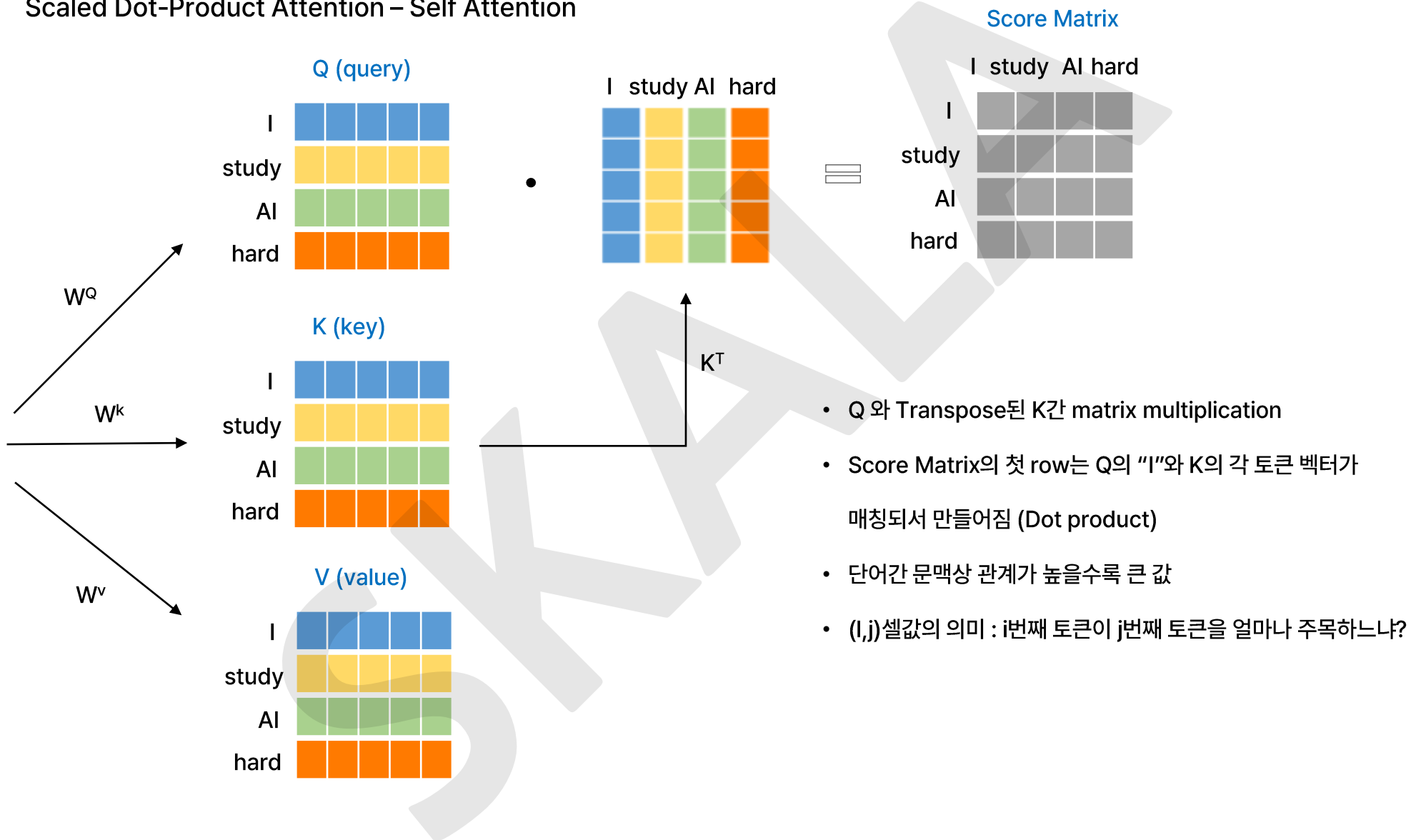
Transformer Encoder

Scaled Dot-Product Attention – Self Attention



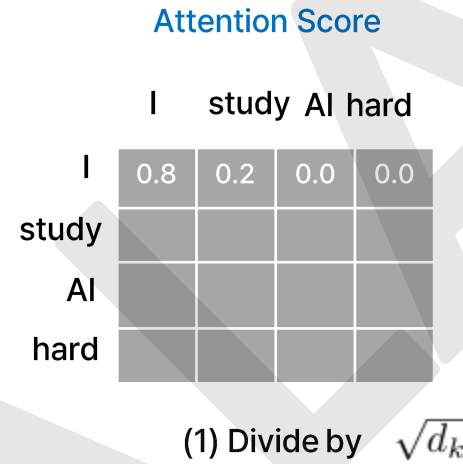
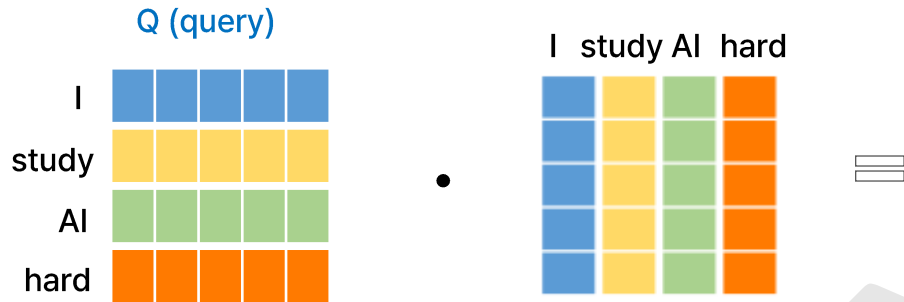
Transformer Encoder

Scaled Dot-Product Attention – Self Attention

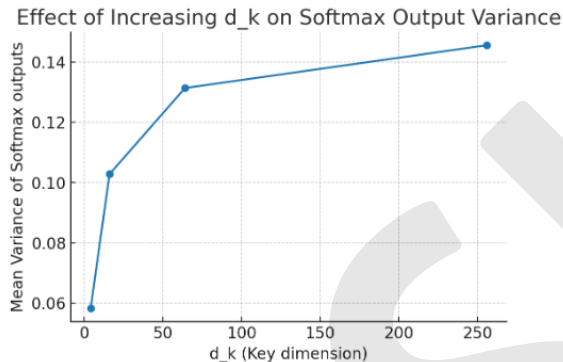


Transformer Encoder

Scaled Dot-Product Attention – Self Attention



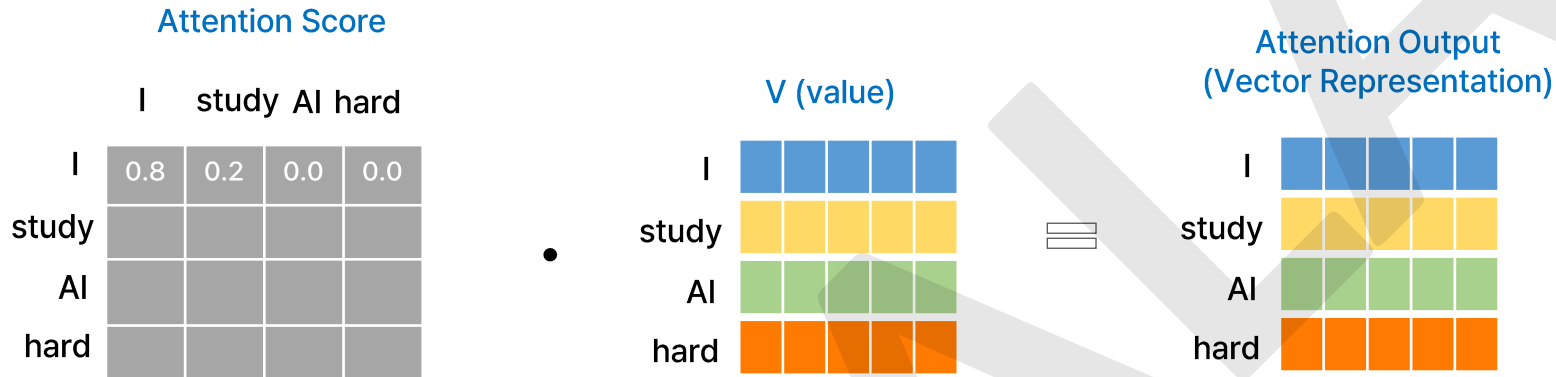
(2) Softmax (확률화)



- Q, K 벡터 차원(d_k)이 커질수록
내적 값의 분산이 커져서,
Softmax 함수의
기울기(gradient)가 매우
작아지는 문제 해소하기 위해 K
차원수의 제곱근으로 나누어줌,
일종의 Scaling
- 이후 Softmax 를 통해 확률화를
진행
- 이로써 내적 값의 크기 범위가
안정화되고, Softmax가 적당한
기울기를 유지

Transformer Encoder

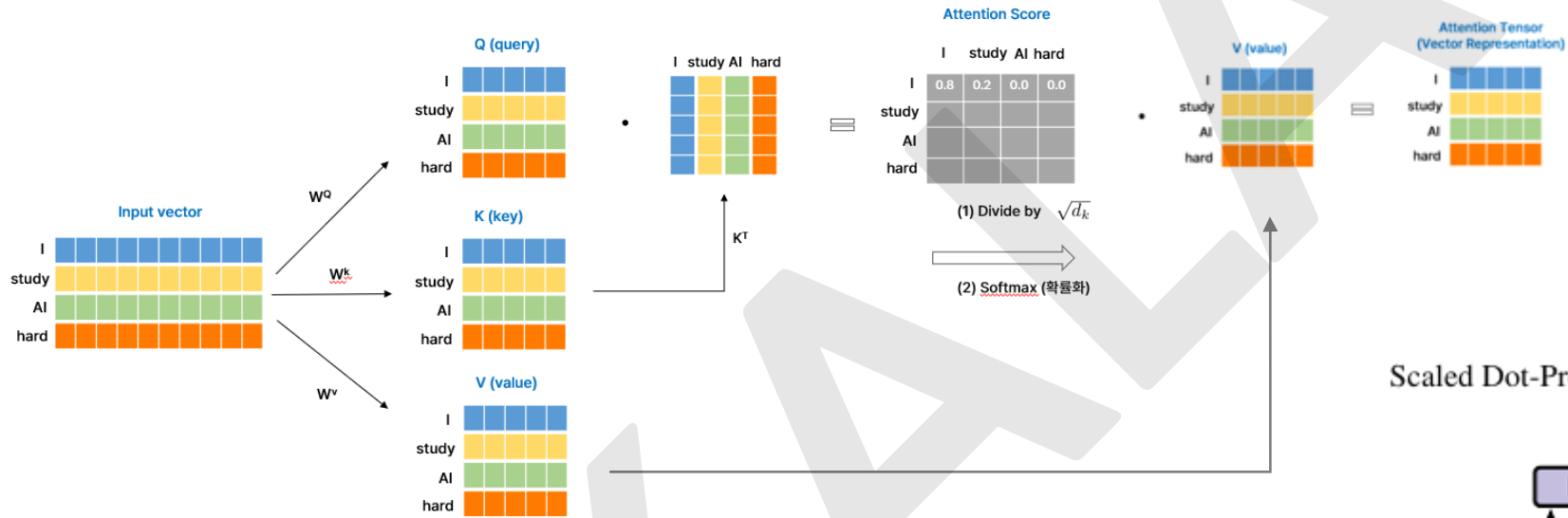
Scaled Dot-Product Attention – Self Attention



- 토큰간 연관성(유사도)를 담고 있는 Attention Score Matrix는 각 토큰이 다른 토큰을 얼마나 주목해야 하는지에 대한 가중치 행렬
- 이에 실제 정보(V)를 행렬곱하여 문맥 벡터를 생성 = 각 토큰이 다른 토큰들의 정보를 비율대로 섞어 만든 새 표현

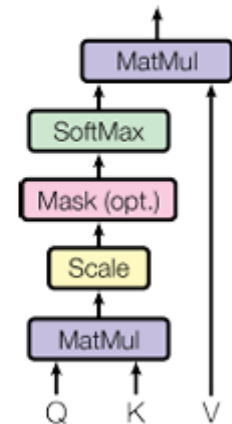
Transformer Encoder

Scaled Dot-Product Attention – Self Attention



$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Scaled Dot-Product Attention



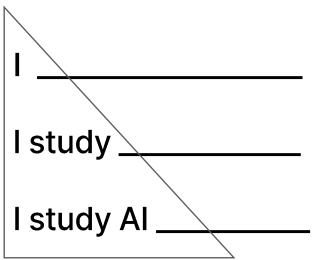
Transformer Decoder

Concept

Problem

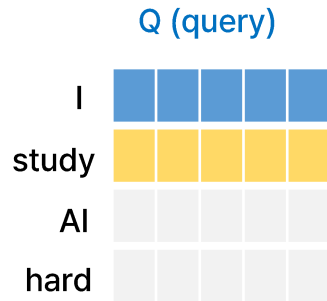
- 어텐션 메카니즘은 모든 단어들이 나와 있을 때 서로 간의 문맥을 파악하는 것이지 그 주어진 문장 그 다음에 토큰을 예상하기 위한 구조는 아님
- 주어진 정보를 가지고 다음 단어를 예측하는 구조로 변경하려면..

Ideation, 셀프어텐션 디코더로 사용하기

- 
- 주어진 토큰들만 가지고 서로를 참조하여 알맞은 피쳐들을 뽑아내고 그를 기반으로 다음 토큰을 예측하도록 한다면..
 - 생성은 주어진 토큰안에서 다음 토큰을 예상하는 것, 그렇게 해보자

Transformer Decoder

Masked Self Attention



K(key) transpose



Attention Score

	I	study	AI	hard
I	1.0	X	X	X
study	0.3	0.7	X	X
AI	0.1	0.3	0.6	X
hard	0.3	0.2	0.1	0.4

- I _____
- I study _____
- I study AI _____

Learning

- 주어진 토큰들 사이의 문맥을 학습하여 다음 토큰 예측
- 이때 Y는 다음 토큰
- Training step마다 Attention Score와 Ground Truth 와 비교하여 Loss 계산

sLLM 구현 및 Fine Tuning

How LLM Learns

- From Raw Text to Training Data
- Context Length vs Attention Cost
- LLM은 다음 토큰을 어떻게 선택할까?
- Cross-Entropy Loss : 예측에서 학습으로

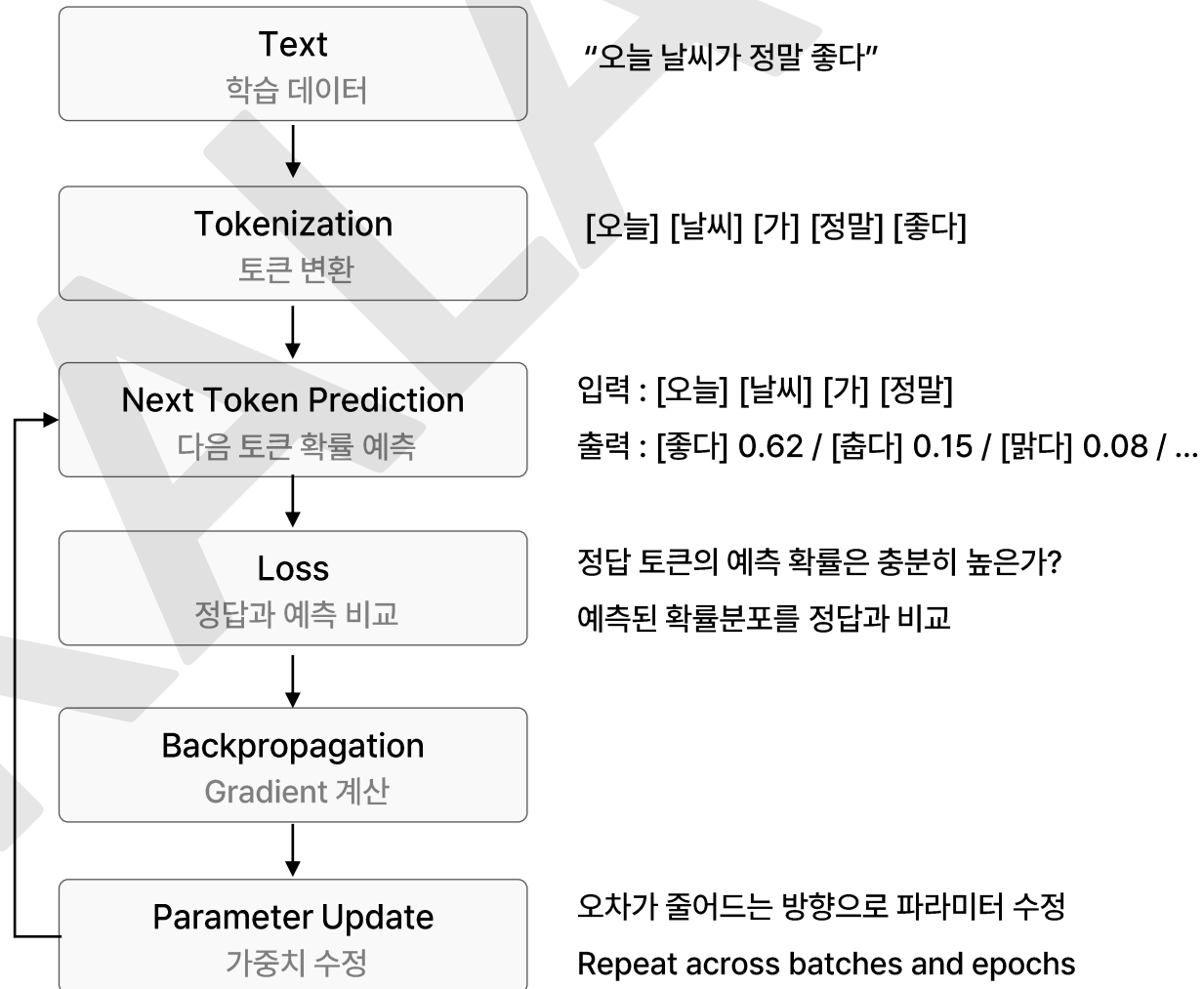
본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

How Does an LLM Learn?

The Big Picture of LLM Training

LLM Learning

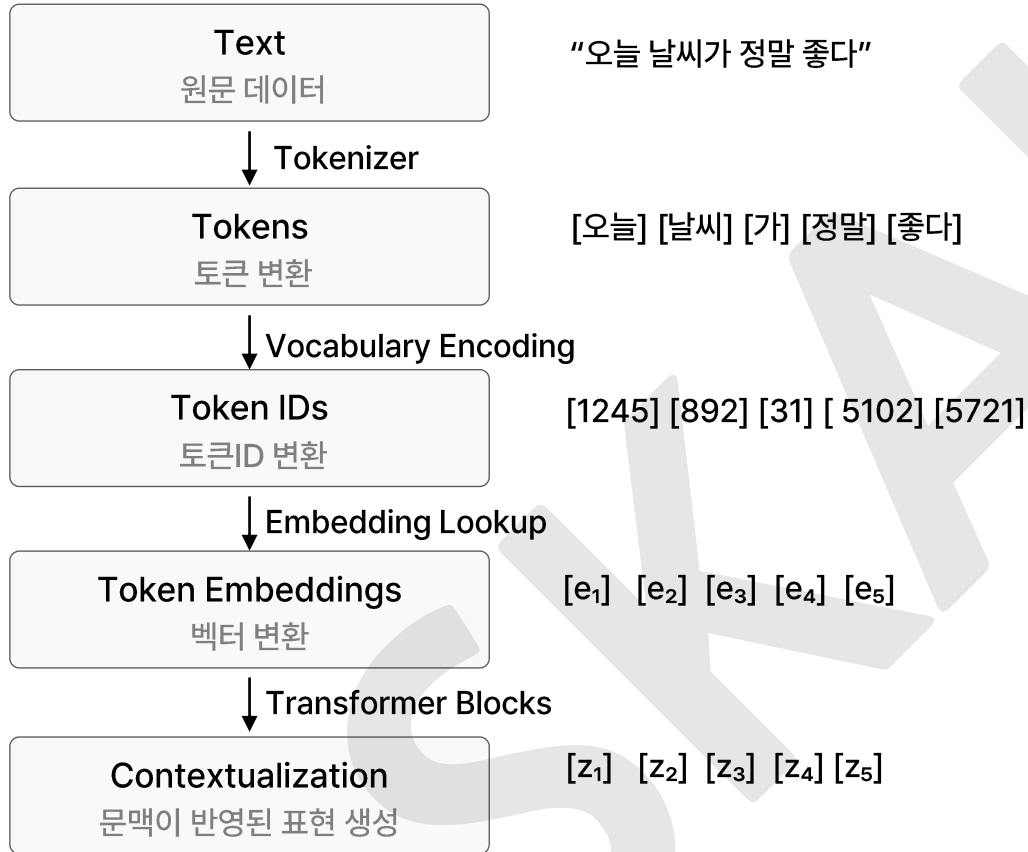
- 다음 토큰을 더 잘 예측하도록 오차를 계산하고, Parameter를 반복적으로 업데이트하는 과정
- Predict → Compare → Update → Repeat



✓ LLM은 예측의 오차로부터 학습

Transformer로 들어가는 Token의 여정

From Text to Contextual Representations



- ✓ Tokenizer는 텍스트를 Token으로 분할하고 각 Token을 Vocabulary의 ID로 변환
- ✓ 모델 내부에서는 각 Token ID에 대응하는 Embedding Vector를 조회
- ✓ Transformer Blocks이 주변 Token과의 관계를 반영하여 Contextual Representation을 생성

From Raw Text to Training Data

텍스트는 어떻게 학습 데이터가 되는가?

Training Data Sample

Raw Text = "오늘 날씨가 정말 좋다"

Position	1	2	3	4
Input	[오늘]	[날씨]	[가]	[정말]
Label	[날씨]	[가]	[정말]	[좋다]
Input IDs	[1245]	[892]	[31]	[5102]
Label IDs	[892]	[31]	[5102]	[5721]

How Training Samples Are Created

- Tokenizer는 원문을 Token Sequence로 변환
- Input의 각 Token 다음에 오는 Token을 Label로 구성
- 따라서 Input과 Label은 한 Token만큼 Shift된 관계
- "각 위치의 Label은 바로 다음 Token"

Causal Language Modeling

Causal Mask, 전체 문장을 입력하면서 미래 Token을 보지 않는 방법

Causal Language Modeling

”

무엇을 학습할 것인가?

Position	Input(실제로 참조할 수 있는 문맥)	Label
1	[오늘]	[날씨]
2	[오늘] [날씨]	[가]
3	[오늘] [날씨] [가]	[정말]
4	[오늘] [날씨] [가] [정말]	[좋다]

- 이전 Token을 바탕으로 다음 Token을 예측하는 학습 방식

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

Causal Mask

”

어떤 정보를 볼 수 있는가?

✓ : Attention 허용, × : 차단

Q\K	오늘	날씨	가	정말	좋다
오늘	✓	×	×	×	×
날씨	✓	✓	×	×	×
가	✓	✓	✓	×	×
정말	✓	✓	✓	✓	×
좋다	✓	✓	✓	✓	✓

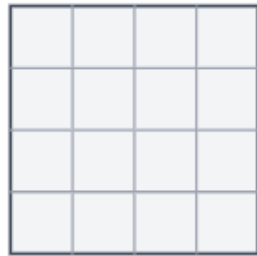
- 학습 시 전체 Token Sequence를 한 번에 입력하지만, Causal Mask가 각 위치에서 미래 Token을 참조하지 못하도록 제한

Context Length vs Attention Cost

Why Attention Becomes a Bottleneck

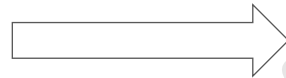
4 × 4 Attention Matrix

Context Length: 4 Tokens



$$4 \times 4 = 16$$

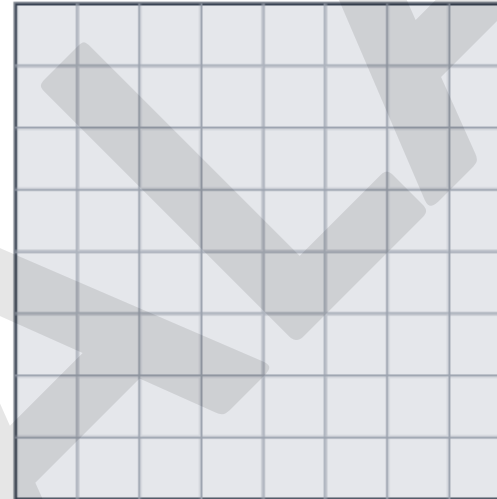
Attention Pairs



Context 2×

8 × 8 Attention Matrix

Context Length: 8 Tokens



$$8 \times 8 = 64 \text{ Attention Pairs}$$

- Self-Attention은 각 Token과 다른 모든 Token 간의 관계를 계산
- 따라서 Context Length가 T일 때 Attention Score Matrix의 크기는 T×T
- Context가 길어질수록 연산량과 메모리 사용량이 빠르게 증가
- 이는 Long-Context LLM의 주요 병목 중 하나가 됨

Context Length	Attention 관계 수	상대 비용
1K tokens	약 1M	1×
2K tokens	약 4M	4×
4K tokens	약 16M	16×
8K tokens	약 64M	64×

Context Length vs Attention Cost

Causal Mask를 적용하면 실제 유효 관계는 대략 절반이지만, 여전히 증가 차수는 $O(T^2)$

Causal Attention 연산량 유도

Q\K	X_1	X_2	X_3	X_4	# Ref.
X_1	✓	×	×	×	1
X_2	✓	✓	×	×	2
X_3	✓	✓	✓	×	3
X_4	✓	✓	✓	✓	4

$$1 + 2 + 3 + 4 = 10$$

$$1 + 2 + 3 + \dots + T$$

$$1 + 2 + \dots + T = \frac{T(T+1)}{2}$$

$$\frac{T(T+1)}{2} = \frac{T^2+T}{2} = \frac{T(T+1)}{2} = \frac{1}{2}T^2 + \frac{1}{2}T$$

by 등차수열 합의 공식

T	$\frac{1}{2}T^2$	$\frac{1}{2}T$
10	50	5
1,000	500,000	500
10,000	50,000,000	5,000

- ✓ Causal Mask Attention의 참조 관계는 Full Attention의 약 절반이지만, Context Length에 따른 증가율은 여전히 T^2

$$\approx \frac{1}{2}T^2 \approx O(T^2)$$

by Big-O notation

Teacher Forcing

학습할 때는 실제 Token, 생성할 때는 예측 Token

Concept

- Teacher Forcing은 학습 시 모델이 이전에 예측한 Token이 아니라, 학습 데이터에 있는 실제 정답 Token을 다음 위치의 입력 문맥으로 제공하는 방식

Training: Teacher Forcing

실제 이전 Token 사용

정답 Sequence가 존재

여러 위치의 예측을 병렬 계산

오류가 다음 입력으로 직접 전파되지 않음

모델 예측 : [오늘] → [기온이] <잘못된 예측>

↓

다음 학습 문맥)

[오늘] [날씨가] ○ [오늘] [기온이] ×

Inference: Autoregressive Generation

모델이 생성한 이전 Token 사용

정답 Sequence가 없음

일반적으로 한 Token씩 순차 생성

생성 오류가 이후 문맥에 누적될 수 있음

모델 예측 : [오늘] → [기온이] <잘못된 예측>

↓

다음 토큰 예측)

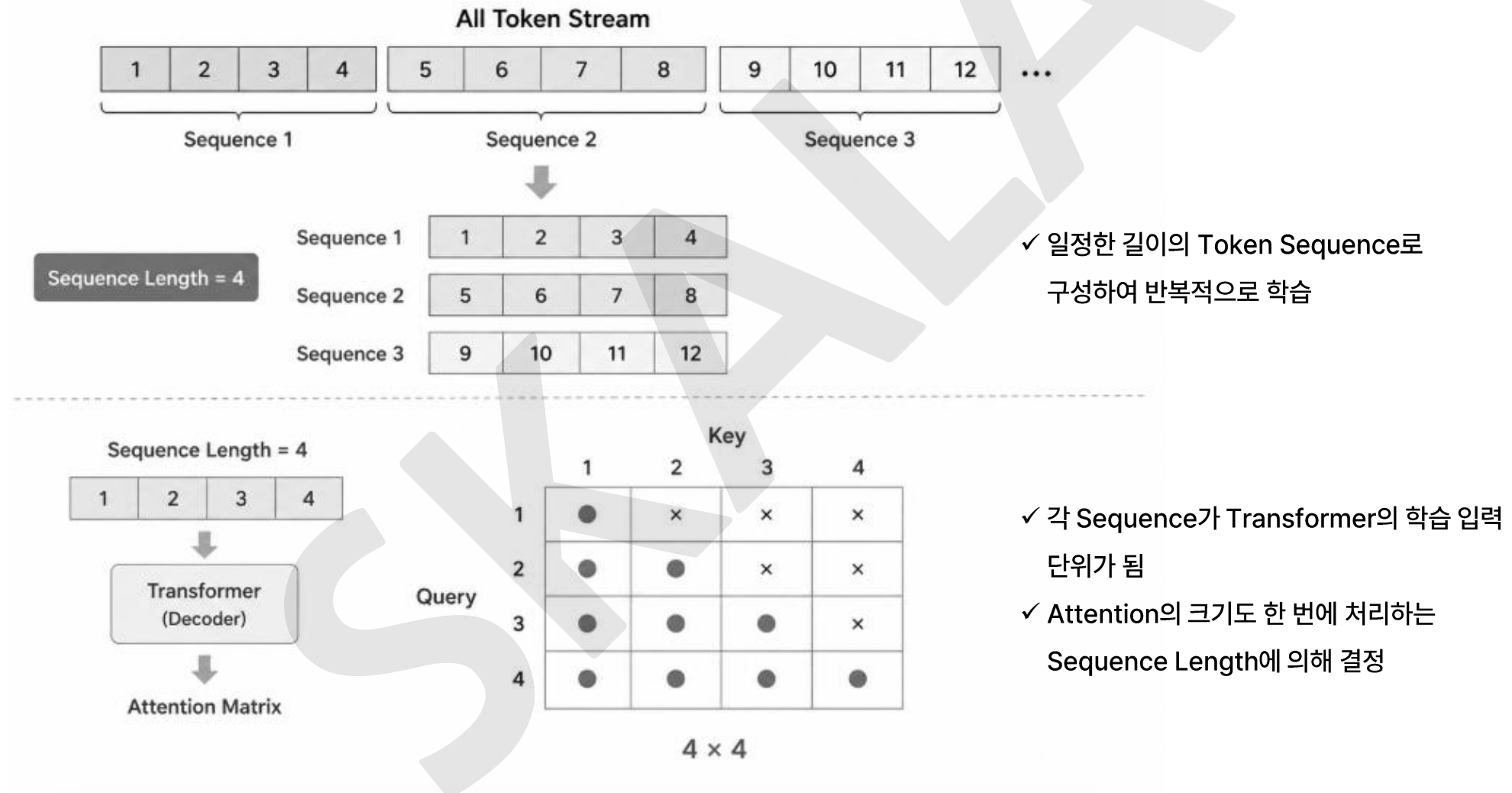
[오늘] [기온이] → 다음 Token 예측

✓ 모델이 잘못 예측했더라도, 다음 위치에는 실제 정답이 포함된 문맥을 제공

✓ 모델이 잘못된 Token을 생성하면 그 Token이 다음 문맥에 포함

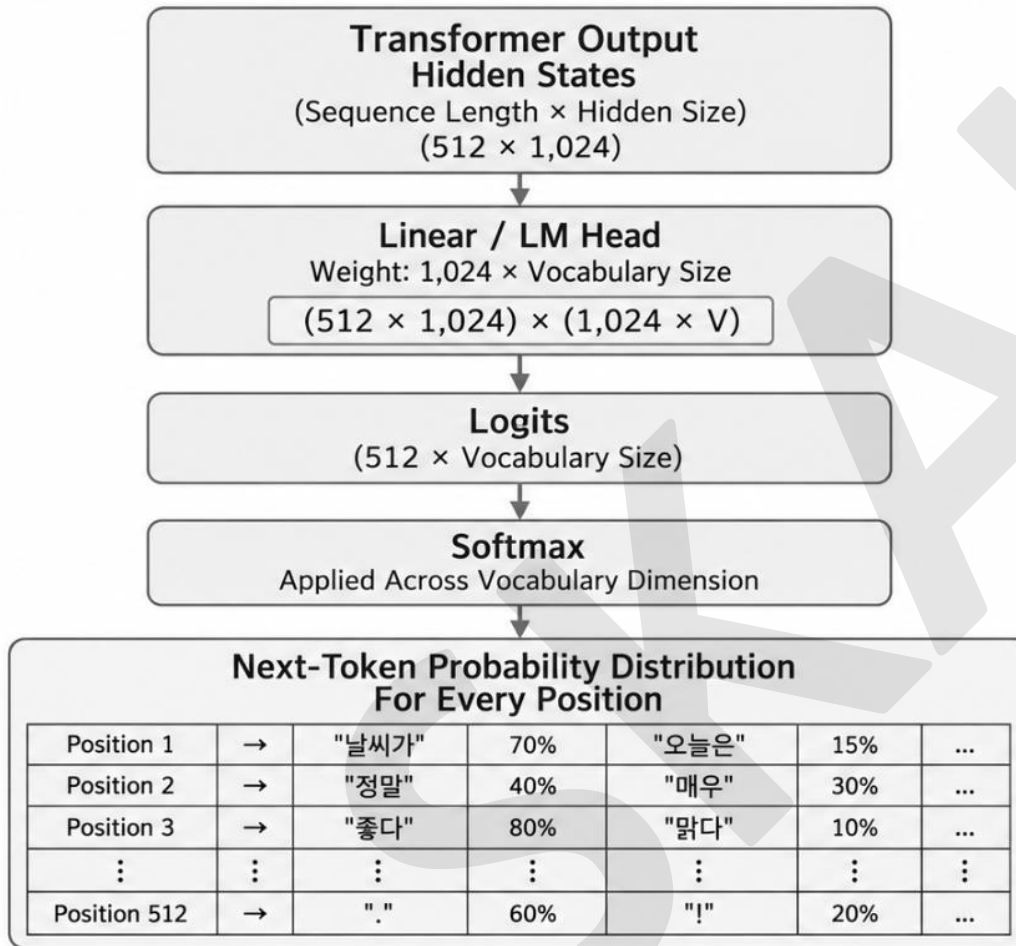
긴 Corpus를 LLM은 어떻게 학습할까?

Long Corpus → Fixed-length Token Sequences → Training



LLM은 다음 토큰을 어떻게 선택할까?

Next Token Probability Distribution

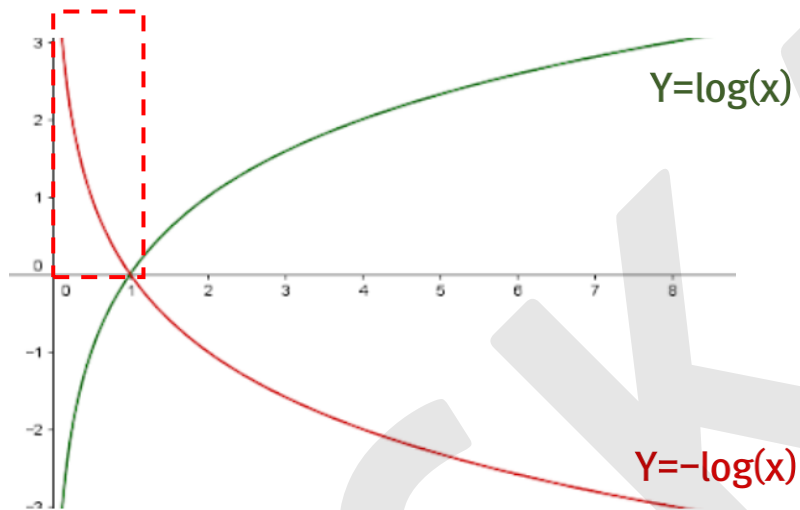


- ✓ Transformer는 모든 토큰 위치에서 다음 토큰의 확률분포를 동시에 계산
- ✓ 각 토큰의 Hidden State를 LM Head가 Vocabulary 크기의 Logit으로 변환

Cross-Entropy Loss : 예측에서 학습으로

높은 정답 확률은 낮은 Loss로, 낮은 정답 확률은 높은 Loss로..

Cross-Entropy Loss:
$$L = - \sum_{i=1}^V y_i \log(p_i)$$



- 100% 확률로 예측했다면 $\rightarrow -\log(1) = 0$
- 80% 확률로 예측했다면 $\rightarrow -\log(0.8) = 0.22$
- 60% 확률로 예측했다면 $\rightarrow -\log(0.6) = 0.51$
- 40% 확률로 예측했다면 $\rightarrow -\log(0.4) = 0.91$
- 20% 확률로 예측했다면 $\rightarrow -\log(0.2) = 1.6$



✓ 확률(확신)이 낮을 수록 loss값(패널티)은 기하급수적으로 증가

Cross-Entropy Loss : 예측에서 학습으로

높은 정답 확률은 낮은 Loss로, 낮은 정답 확률은 높은 Loss로..

Loss 산출 예시

토큰	정답 (y_i)	예측 확률 (p_i)
종다	1	0.80
맑다	0	0.12
춡다	0	0.05
덥다	0	0.03

$$\begin{aligned} L &= -[1 \log(0.80) + 0 \log(0.12) + 0 \log(0.05) + 0 \log(0.03)] \\ &= -\log(0.80) \\ &\approx 0.223 \end{aligned}$$

- ✓ Cross-Entropy Loss는 정답 토큰에 부여한 확률을 예측 오차로 변환
- ✓ 각 위치의 Cross-Entropy Loss를 평균하고, 이를 역전파하여 모델의 파라미터를 업데이트

sLLM 구현 및 Fine Tunning

Fine-Tuning Map

- Fine-Tuning이 필요한 순간
- Fine-Tuning Map : What to Learn
- Fine-Tuning Map : How to Update

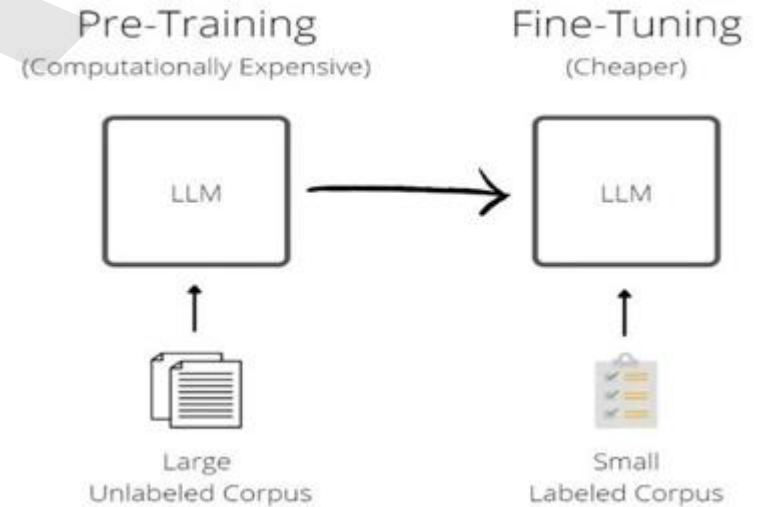
본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Pre-training: 범용 언어 능력의 형성

대규모 텍스트로 범용적인 언어 능력을 형성하는 과정

Concept

- 대규모 텍스트를 사용해 문법·어휘·지식 등 언어 전반의 기초 능력을 학습
- 정답 라벨 없이 텍스트 자체에서 학습 신호를 만드는 자기지도학습(Self-Supervised Learning) 방식
- 다음 토큰 예측을 통해 범용적인 언어 표현과 생성 능력을 형성
- 특정 작업이나 서비스에 최적화된 상태는 아님



What is Fine-Tuning?

Pre-trained Model에 추가 데이터를 학습시켜 특정 목적에 맞게 능력과 행동을 조정하는 과정

Concept

- 이미 사전 학습된 모델에 추가 데이터를 학습시켜 특정 목적에 맞게 모델의 능력과 행동을 조정
- 처음부터 모델을 다시 학습하지 않고, 학습된 언어 능력을 기반으로 Task · Domain · Preference에 적응
- 모델을 실제 업무와 서비스 요구사항에 맞게 조정하는 Model Adaptation 과정

Pre-training	Fine-Tuning
범용적인 언어 능력 형성	특정 목적에 맞게 능력과 행동 조정
대규모 일반 텍스트	상대적으로 작고 목적에 맞게 구성된 데이터
모델의 기초 능력 구축	학습된 능력의 적응과 특화
높은 학습 비용	상대적으로 적은 비용과 시간

Fine-Tuning이 필요한 순간

Prompt와 RAG만으로 일관되게 구현하기 어려운 능력과 행동을 모델에 학습

✔ 특정 작업 능력 강화 (Task Adaptation)

- 범용 모델은 다양한 작업을 수행하지만, 특정 작업에서는 정확도와 일관성이 부족할 수 있음
- 분류·요약·정보 추출·코드 생성 등 반복되는 Task에 특화된 수행 능력을 학습
- ex) 의료 기록의 핵심 항목을 빠짐없이 추출하는 전문 요약

✔ 도메인 언어와 표현 학습 (Domain Adaptation)

- 의료·법률·금융·과학 등 전문 분야의 용어, 표현 방식과 문서 구조를 학습
- 도메인 특유의 문맥과 표현 방식을 이해하고 해당 분야에 적합한 응답을 생성
- ex) 의료 논문의 표현과 문서 구조에 익숙한 도메인 모델

✔ 일관된 응답 행동 학습 (Behavior Customization)

- 응답 형식·문체·절차·정책 등 반복적으로 요구되는 행동 패턴을 학습
- 긴 System Prompt 없이도 일정한 형식과 스타일로 응답하도록 조정
- ex) 고객 문의에 항상 회사의 상담 절차와 지정된 형식, 문체로 응답

Fine-Tuning이 필요한 순간

Prompt와 RAG만으로 일관되게 구현하기 어려운 능력과 행동을 모델에 학습

✔ 작은 모델의 실용적 성능 확보 (Efficient Specialization)

- 대형 범용 모델 대신 작은 모델을 특정 업무에 집중적으로 학습시켜 실용적인 성능 확보
- 반복 호출이 많은 고정 업무에서 추론 비용과 지연시간을 줄일 가능성이 있음
- 온프레미스 환경에서 특정 업무에 최적화된 전용 모델로 활용 가능
- ex) 대형 범용 LLM API → 특정 분류 업무에 특화된 Fine-Tuned sLLM

Why sLLM?

왜 작은 모델을 선택하는가?

Model Choice & Deployment

Privacy·Control·Efficiency

Why Fine-Tuning?

왜 모델을 추가 학습하는가?

Capability & Behavior Adaptation

Adaptation·Specialization·Consistency

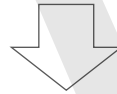
✔ *Fine-Tuned sLLM = Why sLLM? + Why Fine-Tuning?*

Fine-Tuning은 어떻게 설계할까?

”

*What should the model learn,
and how should its parameters be updated?*

”



Fine-Tuning Strategy = Learning Objective × Update Method

Fine-Tuning Map

두 가지 축으로 이해하는 파인튜닝 방법론

”

What to Learn : 무엇을 학습할 것인가?

”

	Continued Pre-training	Supervised Fine-Tuning	Preference Alignment
학습 목적	도메인의 언어와 지식에 적응	원하는 업무와 응답 방식 학습	더 바람직한 답변 기준 학습
학습 데이터	도메인 원문 말뭉치	입력과 모범 답변	선호·비선호 답변 쌍
모델이 배우는 것	전문 용어·표현·문서 패턴	지시 수행·출력 형식·말투	유용성·안전성·응답 품질
대표 방법	Domain-Adaptive Pre-training	Instruction Tuning	RLHF

Continued Pre-training

Fine-Tuning Map : *What to Learn*

Concept

- 별도의 질문과 답변을 만들지 않고, 원문 자체를 학습 데이터로 사용
- 도메인의 용어, 표현, 문서 구조와 지식 분포에 적응
- 기존 Base Model의 다음 토큰 예측 학습을 이어서 수행
- 모델은 다음 내용을 학습
 - 도메인 전문 용어
 - 문장과 문서의 표현 방식
 - 개념 간 관계
 - 도메인에서 자주 등장하는 지식과 패턴

“ 이 분야의 원문을 더 학습하라 ”

금융 보고서 · 의료 논문 · 법률 문서 · 소스 코드 · 사내기술문서

Data Example

수정 듀레이션은 시장금리 변화에 대한채권 가격의 민감도를 나타내는 지표이다.

Supervised Fine-Tuning

Fine-Tuning Map : *What to Learn*

Concept

- 사용자의 입력과 사람이 작성한 모범(정답) 답변을 학습
- 질문-답변, 지시-수행 형태
- 특정 업무 수행 방식과 원하는 출력 형식을 습득
- 모델은 다음 내용을 학습
 - 사용자 지시를 따르는 방법
 - 질문에 답변하는 방식
 - 요약·분류·추출 등의 업무 수행 절차
 - 원하는 말투와 출력 형식
 - 조직이나 서비스의 응답 패턴

“ 이런 요청에는 이렇게 답하라 ”

Data Example

Question:

듀레이션을 초보자에게 설명해 주세요.

Answer:

듀레이션은 금리가 변할 때 채권 가격이 얼마나 움직이는지를 나타내는 지표입니다.

Preference Alignment

Fine-Tuning Map : *What to Learn*

Concept

- 하나의 질문에 대한 여러 답변을 비교하고, 사람이 더 선호하는 답변의 특성을 학습
- 답변 간 비교 또는 평가 데이터를 활용
- 유용성, 정확성, 안전성 등 바람직한 답변의 기준을 학습
- SFT 이후 모델의 응답 품질과 행동을 추가로 정렬
- 모델은 다음과 같은 선호 기준을 학습
 - 더 유용한 답변
 - 더 정확하고 명확한 답변
 - 안전한 답변
 - 불필요하게 장황하지 않은 답변
 - 조직의 정책과 가치에 맞는 답변

“ 여러 답변 중 이런 답변이 더 좋다 ”

Data Example

Question :

감기에 항생제를 먹어야 하나요?

Preference Answer :

감기는 대부분 바이러스성이라 항생제가
중요하지 않습니다. 정확한 판단은 의료진과 상담하세요.

Non-Preference Answer :

항생제를 복용하면 빨리 좋아질 수 있습니다.

Fine-Tuning Map

두 가지 축으로 이해하는 파인튜닝 방법론

”

How to Update : 파라미터를 어떻게 학습할 것인가?

”

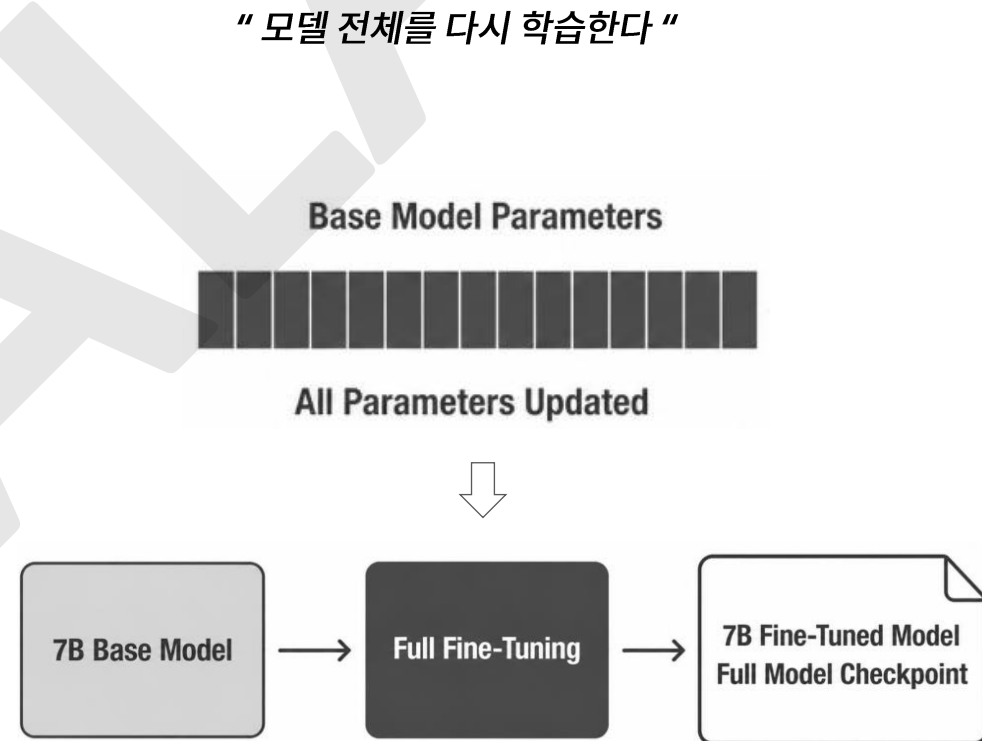
	Full Fine-Tuning	PEFT (Parameter-Efficient Fine-Tuning)
업데이트 범위	모델의 전체 파라미터	일부 파라미터 또는 추가 모듈
Base Model	전체 가중치 업데이트	대부분의 가중치 고정
학습 자원	매우 큼	상대적으로 작음
저장 용량	모델 전체를 저장	작은 Adapter만 저장
대표 방법	Full Parameter Update	<i>LoRA · QLoRA</i>
적합한 상황	충분한 GPU와 대규모 데이터	제한된 자원에서 빠른 실험과 배포

Full Fine-Tuning

Fine-Tuning Map : *How to Update*

Concept

- 모델의 모든 파라미터를 학습
- 모델의 표현과 행동을 폭넓게 변경 가능
- 높은 GPU 메모리와 학습 비용 필요
- 데이터가 부족하면 기존 능력이 손상될 위험 존재
- 작업별로 변경된 전체 모델을 각각 저장 필요
 - 고객 상담 모델 → 전체 7B 체크포인트 저장
 - 금융 분석 모델 → 전체 7B 체크포인트 저장



[참고] Model Checkpoint

Concept

- 학습 중 특정 시점의 모델 상태를 저장한 파일 또는 파일 묶음
- 일반적으로 다음 정보가 포함될 수 있음
 - 모델의 학습된 가중치, 모델 구조와 설정, Optimizer 상태, Learning-rate, 현재 학습 Step 또는 Epoch
- 이를 저장하면 학습을 중단한 지점부터 다시 시작하거나, 특정 시점의 모델을 평가·배포할 수 있음

표현	강조점
Model	모델 자체 또는 구조와 기능
Model Weights	학습된 파라미터 값
Checkpoint	특정 학습 시점에 저장된 모델 상태
Full Model Checkpoint	전체 모델 파라미터를 포함한 저장 결과
Adapter Checkpoint	LoRA 등 추가 학습 파라미터만 저장한 결과

- ✓ Full Fine-Tuning은 목적별로 전체 모델 가중치를 포함한 체크포인트를 저장하는 반면,
- ✓ LoRA는 동일한 Base Model을 공유하고 작은 Adapter Checkpoint만 별도로 저장

Full Fine-Tuning

Fine-Tuning Map : *How to Update*

When to Use?

- 1) 학습·저장 비용보다 최고의 성능이 중요한 경우
 - 모든 파라미터가 자유롭게 변할 수 있어, 더 큰 성능 향상을 낼 가능성이 있음
 - 코드·수학처럼 새로운 복잡한 능력을 학습하는 연구에서는 Full Fine-Tuning이 PEFT보다 정확도와 데이터 효율성이 높았다고 보고
- 2) 전체 파라미터를 안정적으로 학습할 충분한 데이터가 있는 경우
 - 대규모 SFT 데이터로 Base Model에 지시문 이해, 대화 형식 학습, 추론 능력을 본격적으로 학습시키는 경우 (Base Model → Instruction Model)
- 3) 모델을 광범위하게 변화시켜야 하는 경우
 - 의료·법률·금융·과학처럼 기존 모델의 언어 분포와 크게 다른 대규모 말뭉치를 학습시켜, 모델의 내부 표현과 도메인 이해 자체를 바꾸려는 경우 (Continued Pre-training)
 - Meta의 의료 특화 모델 Meditron 사례 : SFT & Full-FT
- 4) 모델 규모가 상대적으로 작고 자원이 충분할 때
 - 수억~수십억 파라미터 규모의 sLLM은 Full Fine-Tuning 비용이 현실적일 수 있음
 - 이런 경우 하나의 완성된 전용 모델을 만드는 선택이 가능

✓ 사라진 방법이 아니라, 최대 성능과 근본적인 모델 변화가 필요할 때 선택하는 방법

PEFT, Parameter-Efficient Fine-Tuning

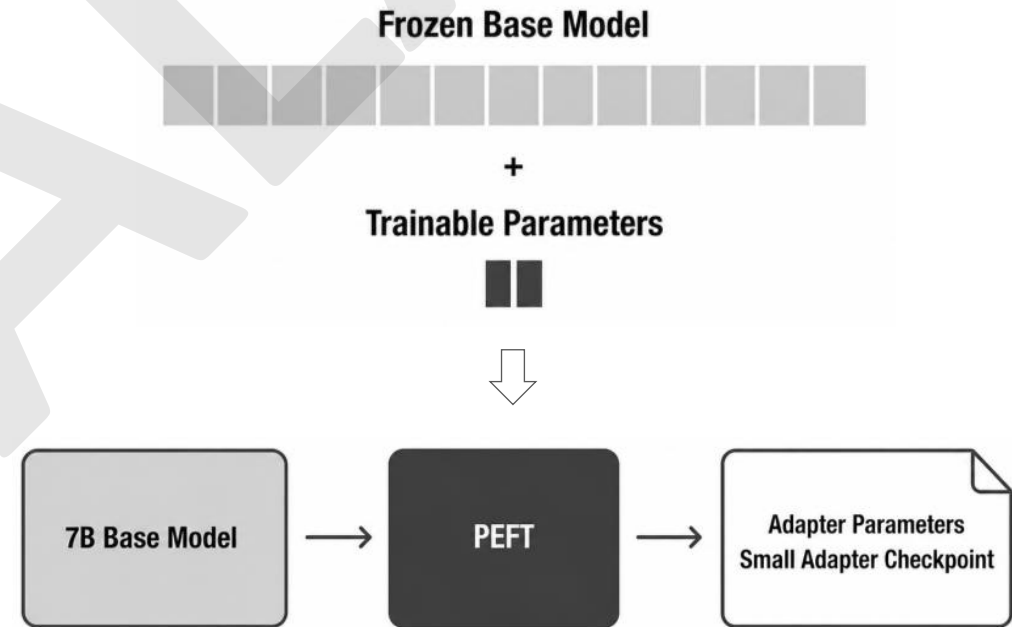
Fine-Tuning Map : *How to Update*

Concept

- 기존 모델의 대부분을 동결(Freeze)
- 소수의 파라미터 또는 추가된 Adapter만 학습
- 학습 메모리와 저장 비용을 크게 절감
- 하나의 Base Model에 여러 Adapter를 교체하여 활용 가능
- 제한된 GPU 환경에서 실무적으로 널리 사용

✓ PEFT의 핵심은 모든 파라미터를 바꾸는 것이 아니라, 필요한 변화를 얼마나 효율적으로 학습하는가에 있다

"Base Model은 고정하고, 작은 부분만 학습한다"



PEFT, Parameter-Efficient Fine-Tuning

Fine-Tuning Map : *How to Update*

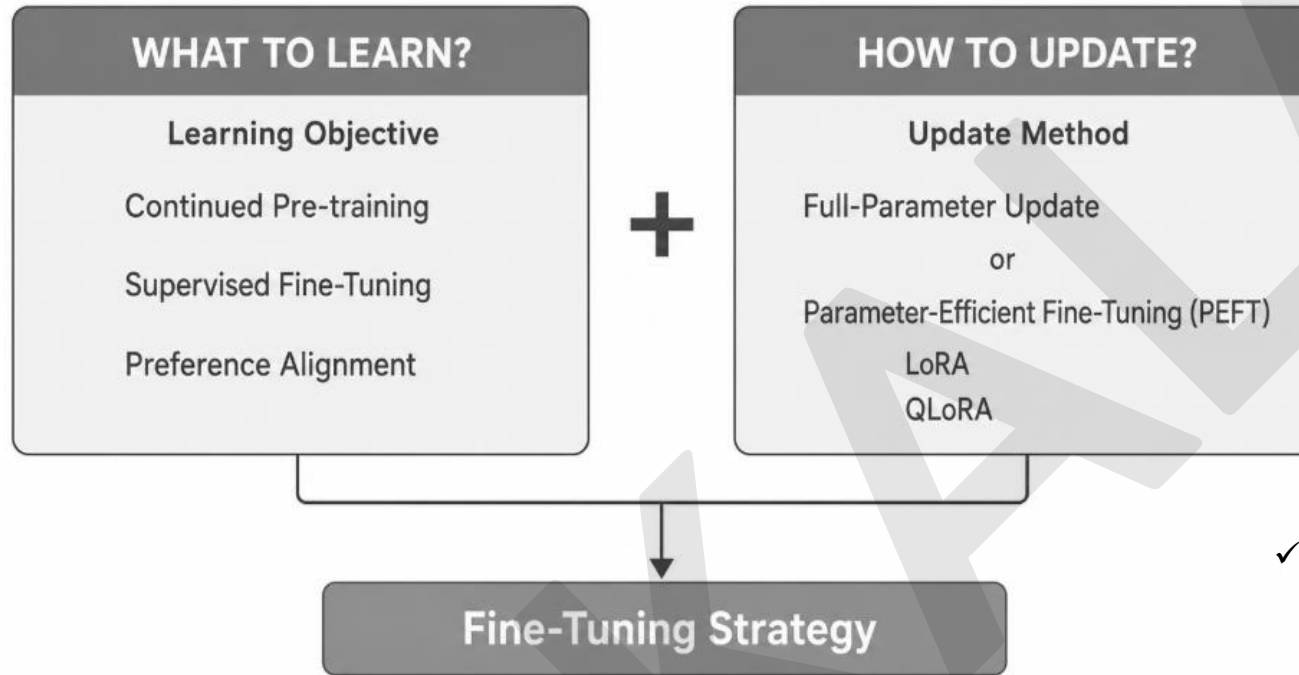
When to Use?

- 1) 학습·저장 비용을 줄여야 하는 경우
 - 일부 파라미터만 학습하므로 GPU 메모리와 학습 비용을 크게 줄일 수 있음
 - 전체 모델이 아닌 작은 Adapter만 저장하므로 체크포인트 저장 및 배포 부담이 작음
- 2) 제한된 자원으로 빠르게 모델을 맞춤화하는 경우
 - Full Fine-Tuning이 어려운 단일 GPU 또는 제한된 환경에서도 대규모 모델의 튜닝이 가능
- 3) 하나의 Base Model을 여러 Task에 활용하는 경우
 - 공통 Base Model은 그대로 유지하고, 고객 상담·문서 요약·코드 생성 등 Task별 Adapter만 별도로 학습
 - 서비스 목적에 따라 Adapter를 교체할 수 있어 여러 모델을 효율적으로 운영할 수 있음
- 4) 반복적인 실험과 업데이트가 필요한 경우
 - 학습 시간이 짧고 체크포인트가 작아 데이터·학습 설정을 바꾸며 빠르게 실험하기에 적합
 - 새로운 데이터나 업무가 추가될 때 전체 모델을 다시 학습하지 않고 Adapter만 업데이트할 수 있음

✓ 효율성·유연성·확장성이 중요할 때 PEFT가 실용적인 기본 선택

What to Learn × How to Update

Fine-Tuning은 '무엇을 학습할 것인가'와 '어떻게 파라미터를 업데이트할 것인가'의 조합으로 설계

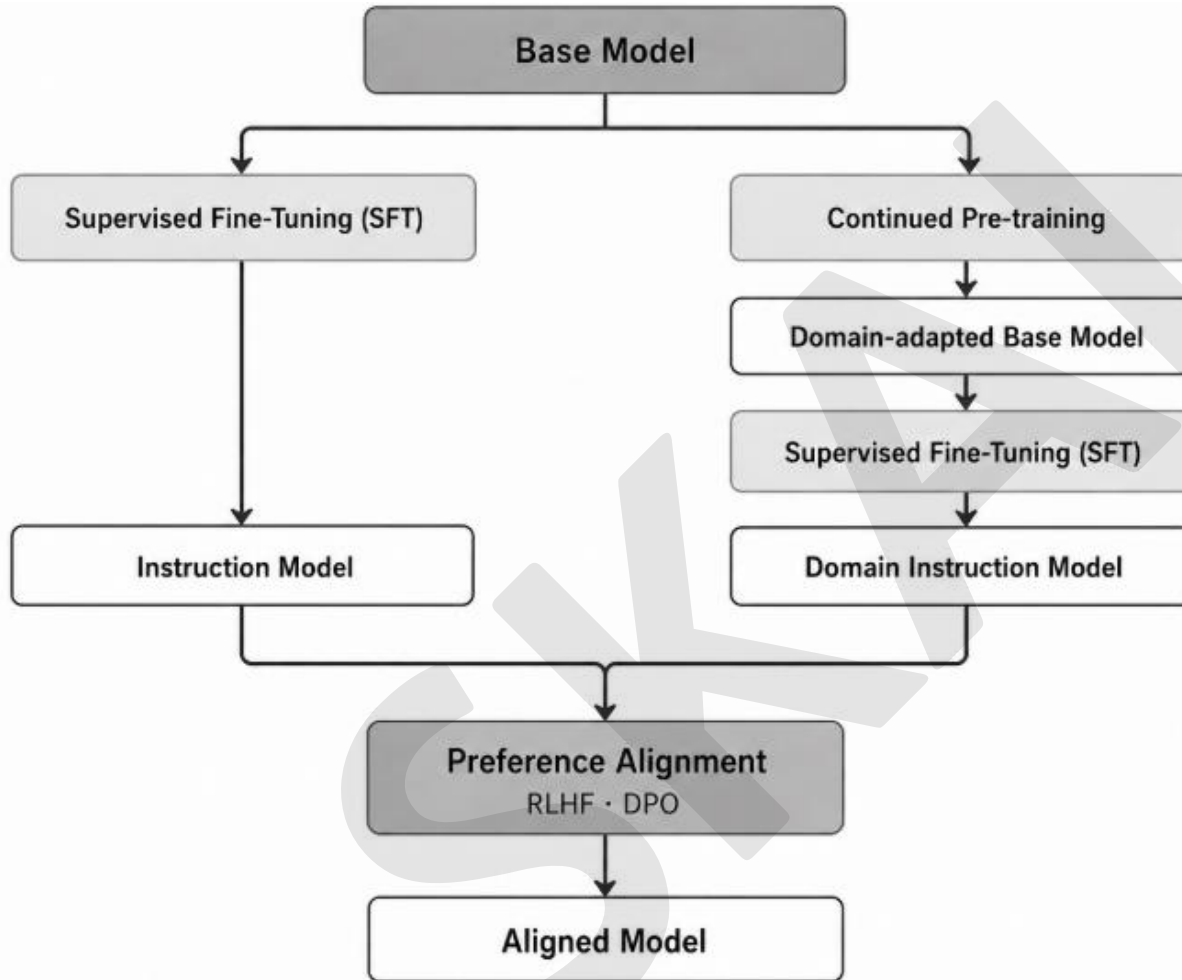


✓ 학습목적이 같아도 업데이트 방식은 달라질 수 있고, 같은 업데이트 방식도 여러 학습 목적에 적용할 수 있음

Learning Objective	Update Method	Resulting Strategy
Continued Pre-training	Full-Parameter Update	Full-Parameter Continued Pre-training
SFT	Full-Parameter Update	Full-Parameter SFT
SFT	QLoRA	QLoRA-based SFT
Preference Alignment	LoRA	LoRA-based Alignment

From Base Model to Aligned Model

Fine-Tuning 기법은 양자택일이 아니라, 목적에 따라 학습 단계를 선택하고 조합하는 과정



- ✓ SFT와 Continued Pre-training은 경쟁 관계가 아니라 순차적으로 조합할 수 있는 학습 단계
- ✓ Continued Pre-training은 선택적으로 거치는 단계이며, 이후 SFT로 연결할 수 있음
- ✓ 모델 개발은 하나의 고정된 경로가 아니라, 목적에 따라 학습 단계를 선택하고 조합하는 과정

sLLM 구현 및 Fine Tuning

Fine-Tuning Techniques

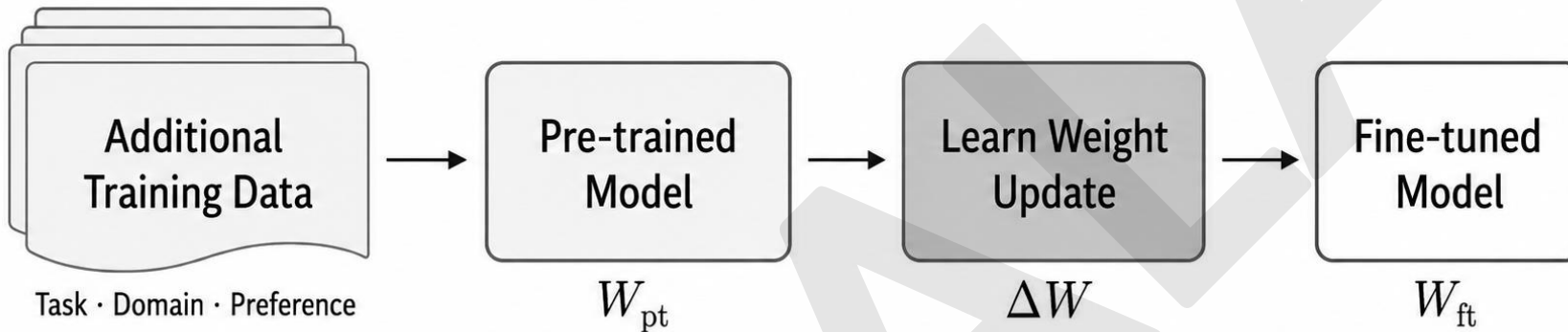
- LoRA
- QLoRA

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Fine-Tuning에서는 무엇이 바뀌는가?

Fine-Tuning learns the required weight update, ΔW

Fine-Tuning as a Weight Update



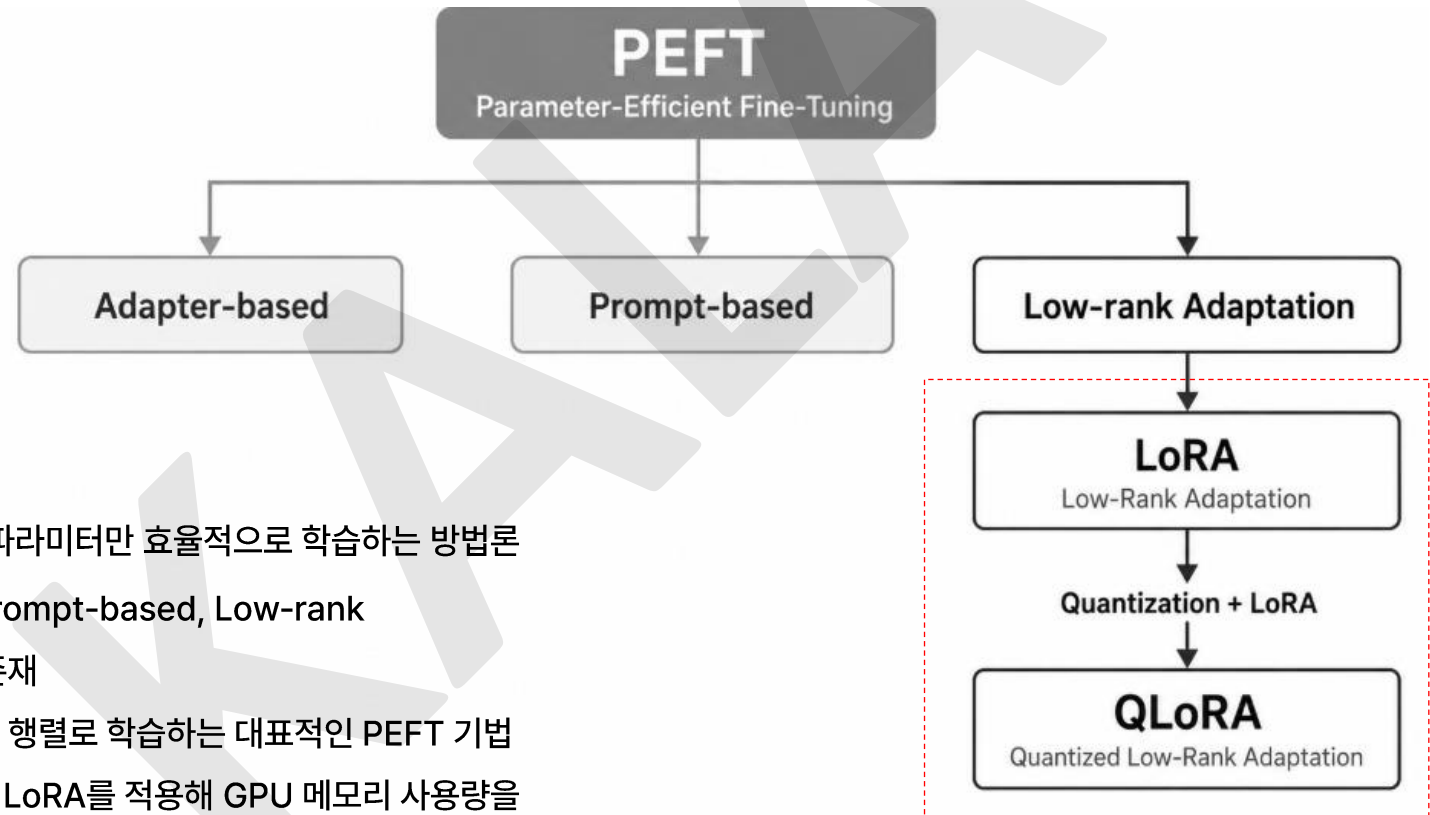
$$W_{\text{fine-tuned}} = W_{\text{pre-trained}} + \Delta W$$

How should ΔW be learned?

- Full Fine-Tuning : 전체 가중치를 직접 업데이트
- PEFT : Base Model을 동결하고 효율적인 구조로 작은 변화량 학습

PEFT Techniques

From LoRA to QLoRA



Overview

- PEFT는 전체 모델이 아닌 일부 파라미터만 효율적으로 학습하는 방법론
- PEFT에는 Adapter-based, Prompt-based, Low-rank Adaptation 등 다양한 접근이 존재
- LoRA는 가중치 변화량을 저차원 행렬로 학습하는 대표적인 PEFT 기법
- QLoRA는 양자화된 기반 모델에 LoRA를 적용해 GPU 메모리 사용량을 더욱 줄이는 방법

LoRA의 Motivation

모델 전체가 아닌, 필요한 변화만 학습할 수 있을까?

Key Idea - Intuitive View

- 정말 특정 Task를 위해 모델의 모든 파라미터를 업데이트해야 할까?
- Task에 필요한 변화만 작게 학습할 수 있을까?
- 모델의 파라미터는 매우 많지만,

특정 Task에 필요한 변화는 몇 가지 핵심 패턴으로 표현할 수 있지 않을까?

Key Idea - Mathematical View

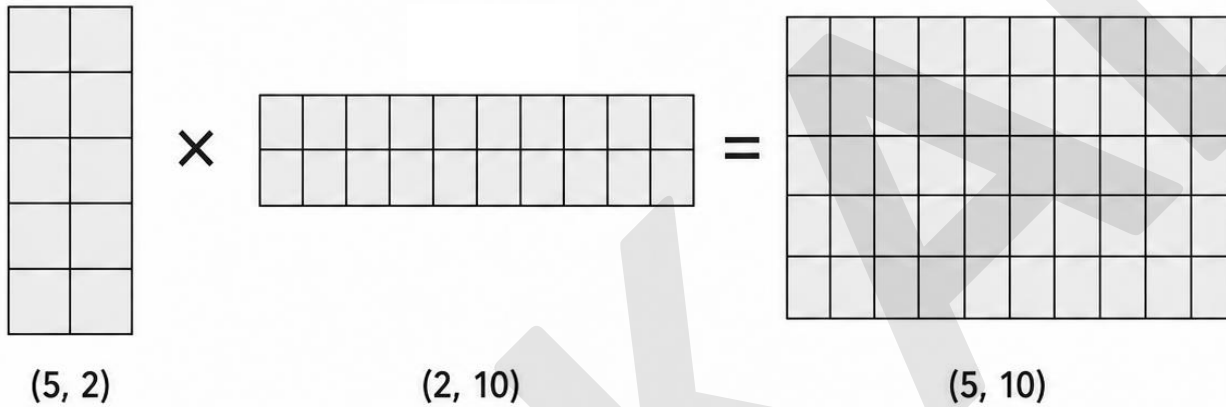
- 특정 Task에 필요한 변화는 전체 파라미터 공간보다 훨씬 낮은 차원의 부분공간에 존재할 수 있지 않을까?
- 그렇다면 기존 가중치는 고정하고, 가중치 변화량만 Low-rank 행렬로 표현해 학습할 수 있지 않을까?

Math Essentials for LoRA

Quick Linear Algebra Refresher

Matrix Multiplication & Shape

- 중간 차원이 일치하면 두 행렬을 곱해 원하는 Shape의 행렬을 만들 수 있음



Math Essentials for LoRA

Quick Linear Algebra Refresher

Rank

- 행렬 W 는 입력 x 를 출력 y 로 변환하는 선형변환

$$y = Wx$$

- Rank는 행렬 W 가 만들어낼 수 있는 선형독립적인 출력 방향의 최대 개수
- Rank가 2라면 모든 출력은 두 개의 독립적인 방향의 조합으로 표현

$$Wx = c1(x)w1 + c2(x)w2$$

$$W = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} | & | & | \\ w_1 & w_2 & w_3 \\ | & | & | \end{bmatrix}$$

$$w_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad w_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad w_3 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

$$w3 = w1 + w2$$

Math Essentials for LoRA

Quick Linear Algebra Refresher

Rank of a Matrix Product

- 행렬곱은 두 선형변환을 차례로 적용하는 과정
- 앞 단계에서 표현하지 못한 독립적인 방향을 다음 단계가 새롭게 복원할 수 없음
- 따라서 곱의 Rank는 두 행렬 중 더 작은 Rank를 넘을 수 없음

$$B \in \mathbb{R}^{m \times k}, \quad A \in \mathbb{R}^{k \times n} \rightarrow BA \in \mathbb{R}^{m \times n}$$



$$\text{rank}(BA) \leq \min(\text{rank}(B), \text{rank}(A))$$



$$\text{rank}(B) \leq k$$

$$\text{rank}(A) \leq k$$



$$\text{rank}(BA) \leq \min(\text{rank}(B), \text{rank}(A)) \leq k$$

Math Essentials for LoRA

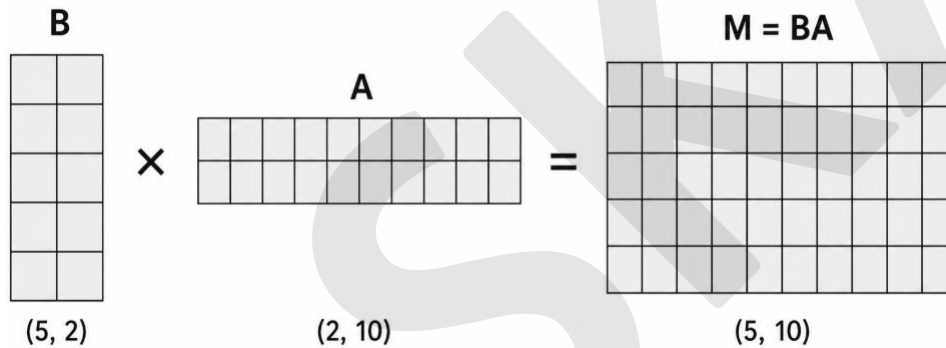
Quick Linear Algebra Refresher

Low Rank Factorization

- 행렬 M 을 중간 차원 r 을 갖는 두 행렬의 곱으로 표현하는 것

$$M = BA, \quad B \in \mathbb{R}^{m \times r}, \quad A \in \mathbb{R}^{r \times n}$$

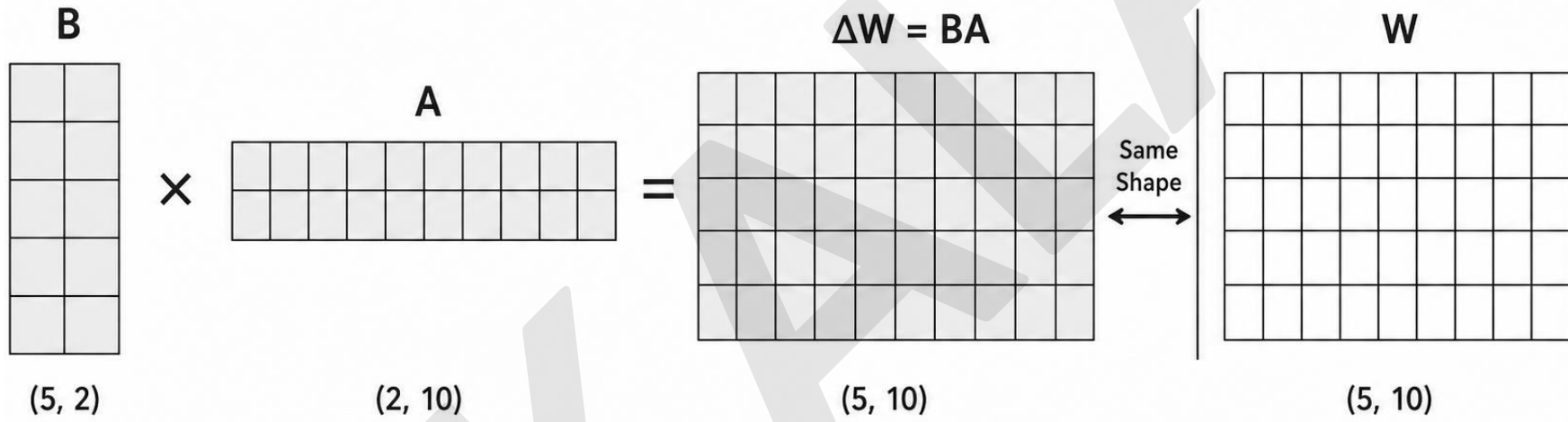
- $r \ll \min(m, n)$ 이면 결과 행렬은 저차원 구조를 가짐
- 이렇게 r 을 제한하여 저차원으로 근사하는 것을 Low-Rank Factorization이라 함
- 중간 차원이 실제 Rank와 같은 경우, 즉 정확히 $r = \text{rank}(M)$ 이면 Rank Factorization이라 함



LoRA의 핵심: Low-Rank Weight Update

From Low-Rank Factorization to LoRA

$$r = 2, \quad \text{rank}(\Delta W) \leq 2$$



LoRA weights

- $r=2$ 로 분해 $\rightarrow (5,2), (2,10)$ Matrix 학습
- Trainable parameters = 30
- Trainable Parameters 40% 감소

Original weights

- $(5,10)$ Matrix 직접학습
- Trainable parameters = 50

LoRA : Low-Rank Adaptation

Pre-trained weight는 고정하고, Low-Rank로 제한된 가중치 변화량만 학습

How LoRA Works

- 사전학습 가중치 W_0 는 고정
- 가중치 변화량을 두 개의 작은 행렬로 구성

$$\Delta W = BA$$

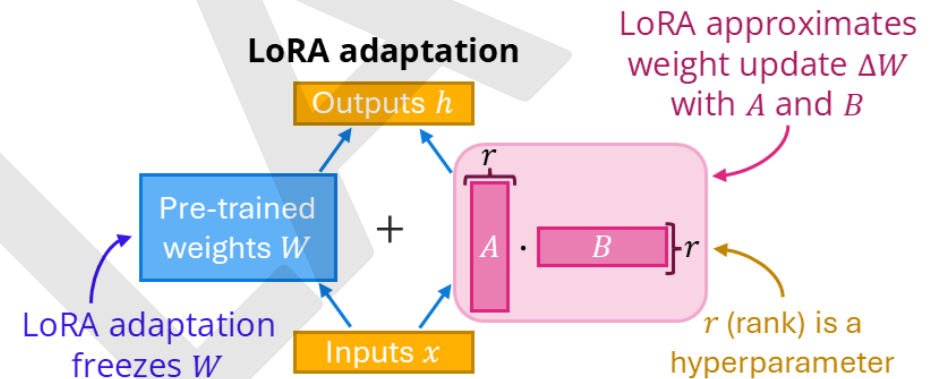
- Fine-Tuning 과정에서는 A,B만 학습

$$h = W_0x + BAx$$

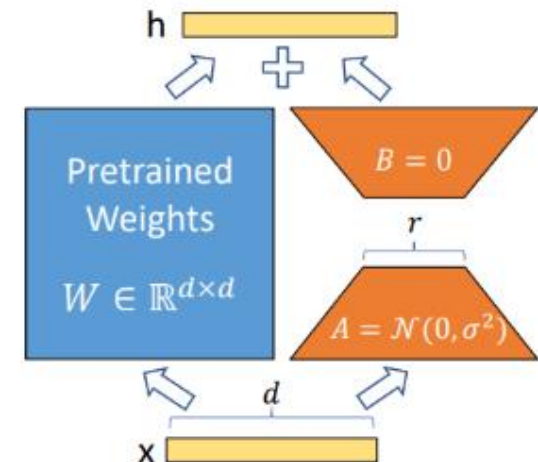
- 중간 차원 r 이 작으므로 학습 파라미터가 크게 감소

$$r \ll \min(d_{\text{in}}, d_{\text{out}})$$

- ✓ Full Fine-Tuning은 W 전체를 업데이트하지만, LoRA는 Task 적응에 필요한 Low-Rank 변화량만 학습



$$h = w_0x + \Delta x = W_0x + BAx$$

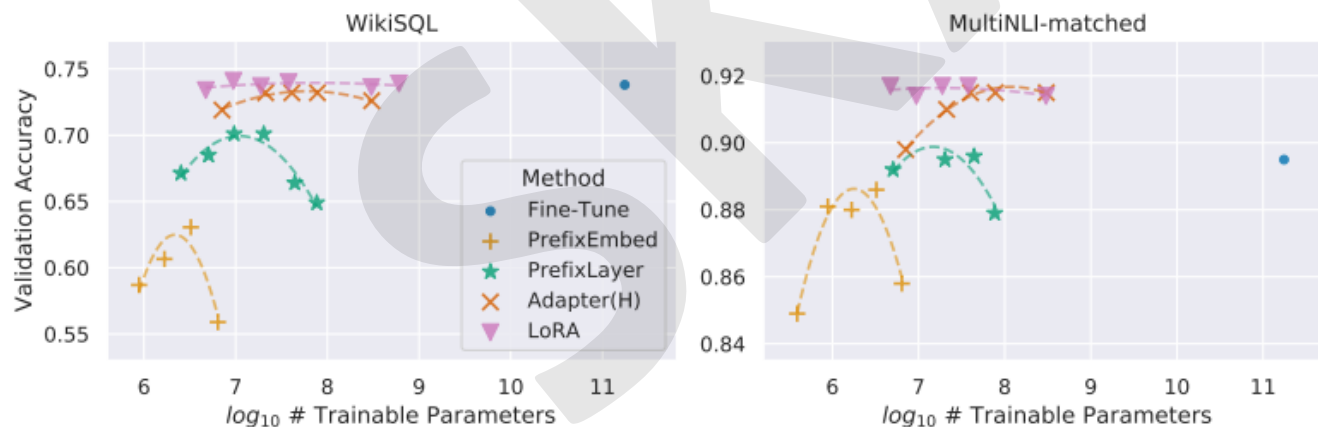


LoRA : Low-Rank Adaptation

LoRA는 Full Fine-Tuning 대비 약 0.02% 수준의 파라미터로 동등하거나 더 우수한 성능을 달성

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.



LoRA Hyper-parameters

핵심 Hyper-parameter 4요소

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
    ],
    bias="none",
    task_type="CAUSAL_LM",
)

model = get_peft_model(
    model,
    lora_config,
)

model.print_trainable_parameters()
```

- ✓ LoRA는 Rank, Scaling, Dropout, 적용 모듈을 설정하여 모델에 Low-Rank 학습 경로를 추가

LoRA Hyper-parameters

핵심 Hyper-parameter 4요소

Hyper-parameter	의미	주요 영향
r	Low-Rank 중간 차원	표현력·학습 파라미터 수
lora_alpha	LoRA 변화량의 스케일	기존 출력에 반영되는 강도
target_modules	LoRA를 적용할 모듈	학습 범위·파라미터 수
lora_dropout	LoRA 경로의 Dropout	과적합 방지·일반화

- ✓ LoRA는 Rank, Scaling, Dropout, 적용 모듈을 설정하여 모델에 Low-Rank 학습 경로를 추가
- ✓ 표현력과 효율성 사이의 Trade-off를 결정

LoRA Hyper-parameters

핵심 Hyper-parameter 4요소

Rank r

- LoRA가 학습할 수 있는 변화 공간의 크기를 결정
- $r \uparrow$: 높은 표현력, 파라미터 증가, 과적합 가능성 증가
- $r \downarrow$: 파라미터 감소, 낮은 메모리 사용, 제한된 표현력
- r 을 두 배로 하면 LoRA 파라미터 수도 거의 두 배

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}$$

$$\text{rank}(\Delta W) \leq r$$

$$N_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}})$$

LoRA Alpha α

- α 는 학습된 Low-Rank 변화량이 기존 모델에 반영되는 강도를 조절
- LoRA가 학습할 수 있는 변화 공간의 크기를 결정
- $\alpha \uparrow$: LoRA 변화량을 강하게 반영
- $\alpha \downarrow$: LoRA 변화량을 보수적으로 반영
- 지나치게 크면 학습이 불안정하거나 기존 능력을 크게 변경할 수 있음
- 유의할 점은 α 자체보다 α/r 의 관계를 보는 것

$$h = W_0 x + \frac{\alpha}{r} B A x$$

$$\text{LoRA Scaling} = \frac{\alpha}{r}$$

LoRA Hyper-parameters

핵심 Hyper-parameter 4요소

Target Modules

- LoRA를 Transformer의 어떤 가중치 행렬에 적용할지 결정
- 많은 모듈에 적용 : 표현력과 학습 파라미터 증가
- 일부 모듈에 적용 : 파라미터와 메모리 절감
- 같은 r이라도 적용 모듈이 많아지면 전체 Trainable Parameters가 크게 증가

W_q, W_k, W_v, W_o

"q_proj", "k_proj", "v_proj", "o_proj"

LoRA Dropout

- LoRA 경로의 입력 일부를 학습 중 확률적으로 제거
- 0.0 : Dropout을 적용하지 않음
- 0.05 : 약한 정규화
- 0.1 : 상대적으로 강한 정규화
- 작은 데이터셋에서 과적합이 우려되면 높이는 것을 검토

$$h = W_0x + \frac{\alpha}{r}BA(\text{Dropout}(x))$$

LoRA Hyper-parameters

Implementation Settings

Hyper-parameter	의미	일반적인 설정
bias	Bias 파라미터도 학습할지 결정	"none"
task_type	모델이 수행할 Task 유형	"CAUSAL_LM"
modules_to_save	LoRA 외에 함께 학습·저장할 모듈	Task에 따라 설정
init_lora_weights	(A,B) 초기화 방식	기본값 사용
use_rslora	Rank-Stabilized LoRA 사용 여부	선택적, True/False

Vanilla

$$\text{Scaling} = \frac{\alpha}{r}$$

Rank-Stabilized

$$\text{Scaling} = \frac{\alpha}{\sqrt{r}}$$

Practical Guide

하이퍼파라미터	가장 일반적인 설정	비고
target_modules	q_proj, k_proj, v_proj, o_proj gate_proj, up_proj, down_proj	Attention + MLP 전체, QLoRA 표준
r (rank)	16	8~32 범위에서 가장 무난한 값
lora_alpha	32	관례적으로 r의 2배
lora_dropout	0.05 ~ 0.1	데이터 적으면 0.1 권장
bias	"none"	학습 파라미터 최소화, 가장 혼함
task_type	"CAUSAL_LM"	언어모델 파인튜닝시 고정값

- r : 4~8 (매우 가벼움, 간단한 태스크), 16 (가장 무난하고 널리 쓰임), 32~64(복잡한 태스크, 데이터 많을 때)
- alpha/r 비율이 실제 스케일링 강도를 결정 (보통 2.0 비율 사용)
- Dropout : 데이터가 적으면 0.1, 많으면 0.05 정도로 낮춰도 무방

Quantization

더 적은 비트로 가중치 표현하기

표현 방식	가중치당 비트	10억 개 가중치
FP32 (32 bit Floating Point)	32-bit	약 4 GB
FP16 (16 bit Floating Point)	16-bit	약 2 GB
INT8 (8-bit Integer)	8-bit	약 1 GB
NF4 (4-bit Normal Float)	4-bit	약 0.5 GB

Effect

- 비트 수가 줄어들수록 메모리 사용량 감소
- 하드웨어와 연산 커널이 지원하는 경우 연산 효율 향상 가능
- 표현 정밀도가 낮아지면서 표현 가능한 값의 수 감소
- 원본 값과 근사값 사이에 Quantization Error 발생할 수 있음

✓ Quantization은 모델의 가중치를 더 적은 비트로 근사하여 메모리 사용량을 줄임

Scalar Quantization

연속적인 실수 값을 어떻게 제한된 수의 값으로 바꿀까?

Concept

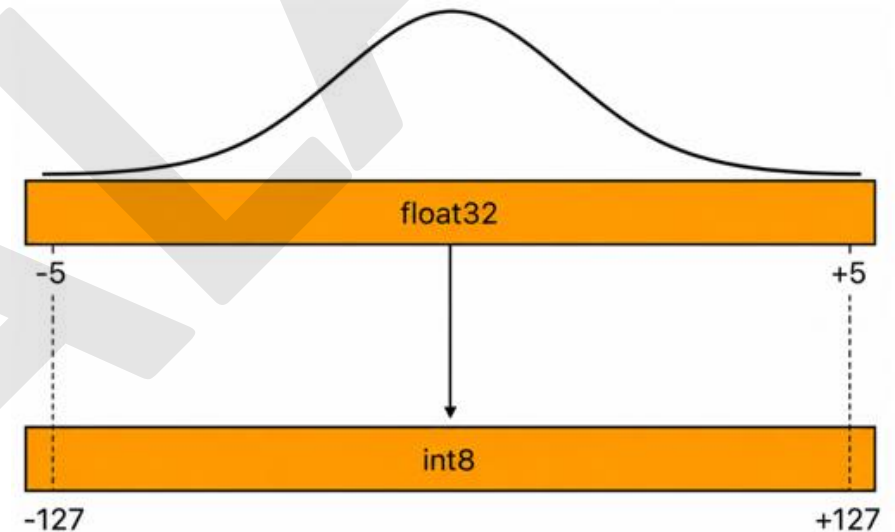
- 벡터의 각 원소를 제한된 개수의 이산적인 값 중 하나로 개별 매핑하여, 더 적은 비트로 근사 표현하는 방식
- 각 원소를 독립적으로 양자화
- 16/32-bit Floating Point 가중치를 INT8 또는 4-bit 등의 저정밀 표현으로 변환

Scalar Quantization

Case 1. Uniform Quantization

- Step 1. **표현 범위 결정 (Range)**
- [3.75, -2.98, 1.43, 0.87, -4.21]
벡터를 탐색하여 양자화할 실수 값의 최소값과 최대값을 결정 → [-5, 5]
- Step 2. **스케일링 (Scaling)**
- 최소값과 최대값을 기준으로 범위를 정규화
 - 실수 범위 [-5, 5]를 INT8 범위 [-127, 127]에 대응 → 실수 값을 일정한 비율로 확대하여 정수 범위로 변환 (8-bit 표현)
- Step 3. **반올림 및 클리핑(Rounding & Clipping)**
- 변환된 값을 가장 가까운 정수로 반올림
 - INT8 범위를 벗어난 값은 -127 또는 127로 제한
 - 최종 값을 8-bit 정수로 저장

$$q = \text{clip} \left(\text{round} \left(x \times \frac{127}{5} \right), -127, 127 \right)$$



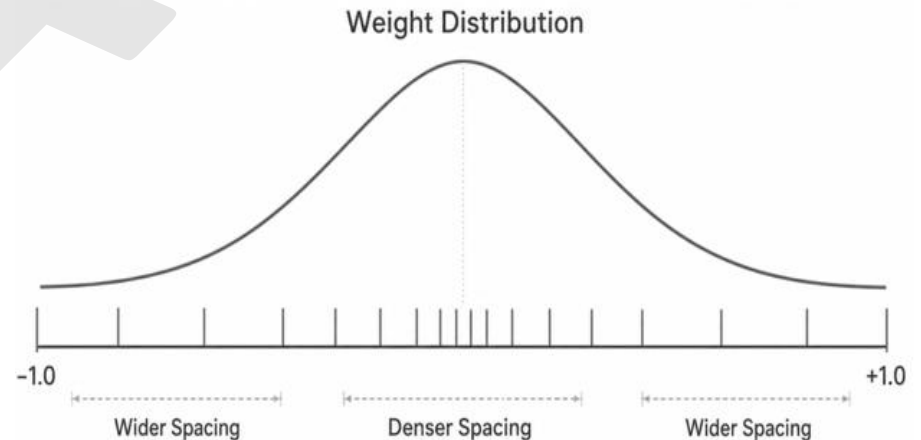
Original (float32)	3.75	-2.98	1.43	0.87	-4.21
Quantized (INT8)	95	-76	36	22	-107

Scalar Quantization

Case 2. NF4 Quantization (Non-uniform)

- Step 1. ○ **Block 분할 및 범위 결정 (Block-wise Range)**
- 가중치를 작은 Block 단위로 분할
 - 각 Block에서 절댓값이 가장 큰 값인 Absmax를 계산
 - Block별로 서로 다른 양자화 범위를 적용
- Step 2. ○ **정규화 (Normalization)**
- Block의 각 가중치를 Absmax로 나눔
 - 모든 값을 $[-1, 1]$ 범위로 정규화
- Step 3. ○ **NF4 값으로 매핑 (Nearest NF4 Mapping)**
- 정규화된 각 가중치를 가장 가까운 NF4 값으로 매핑
 - NF4는 분포를 고려한 16개의 비균등한 값을 사용
 - 선택된 NF4 값의 위치를 4-bit Index로 저장

- ✓ 가중치가 많이 분포하는 영역에 양자화 값을 더 촘촘하게 배치
- ✓ 가중치를 16개의 NF4 값으로 근사하여 4-bit Index로 저장
- ✓ NF4는 적은 수의 값만 사용하면서도, 가중치 분포에 맞게 양자화 값을 배치하여 정보 손실을 줄이는 접근



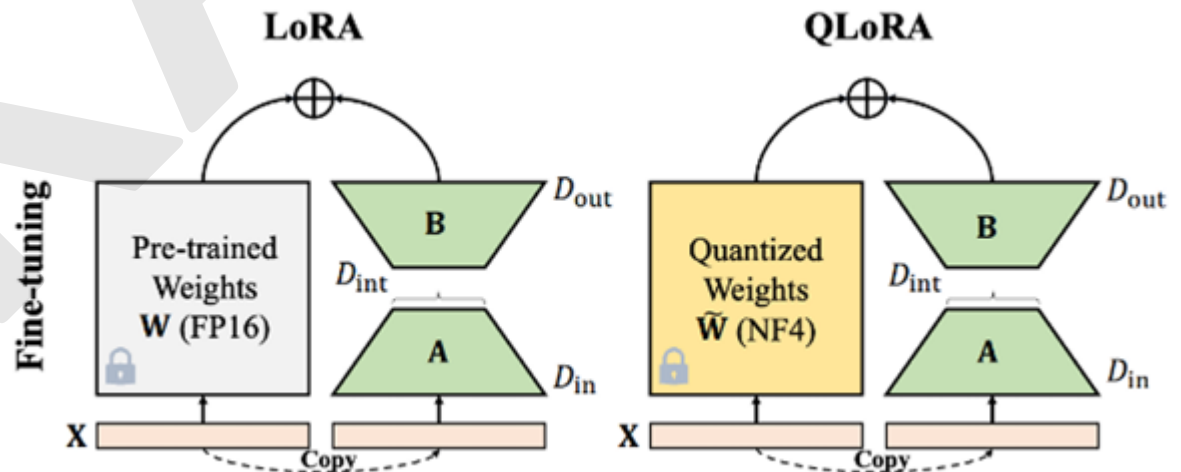
- ✓ 양자화 값은 반드시 일정한 간격으로 배치할 필요가 없다

QLoRA: Quantization + LoRA

Base Model은 작게 저장하고, LoRA Adapter만 학습

Concept

- QLoRA는 Base Model을 4-bit로 저장하고 고정한 상태에서, 작은 LoRA Adapter만 학습하는 방법
 - Base Model의 주요 가중치를 4-bit NF4로 저장하여 GPU 메모리 사용량 절감
 - 학습 과정에서는 저정밀 Base Model을 고정하고, LoRA의 $\Delta W = BA$ 만 학습
 - LoRA의 파라미터 효율성에 Base Model의 저장 메모리 절감을 결합
- ✓ Quantization은 Base Model의 저장 메모리를 줄이고, LoRA는 학습할 파라미터를 줄인다.



QLoRA는 메모리를 어떻게 아끼는가?

저장(NF4) + 압축(Double Quant) + 관리(Paged Optimizer) = 메모리 사용량 최소화

✓ NF4 (NormalFloat 4-bit)

“가중치를 작게 저장한다”

- Base Model의 가중치를 32bit/16bit가 아닌 4bit로 저장
- 신경망 가중치가 정규분포를 따른다는 특성에 최적화된 데이터 타입
- 같은 4bit라도 일반 정수형(INT4)보다 정보 손실이 적음

✓ Double Quantization

“양자화 상수도 다시 압축한다”

- 양자화 과정에는 "양자화 상수(quantization constant)"라는 보조 값이 별도로 저장되는데, 이것 자체도 용량을 차지함
- QLoRA는 이 양자화 상수를 한 번 더 양자화해서 추가로 압축
- 파라미터당 약 0.4bit를 추가 절감 (모델이 클수록 효과 커짐)

✓ Paged Optimizer

“메모리 스파이크를 대비한다”

- 학습 중 특정 순간 메모리 사용량이 급증(spike)해서 GPU 메모리가 부족해지는 경우 발생
- NVIDIA 통합 메모리 기능을 활용해, 넘치는 부분을 CPU 메모리로 자동 이동(paging)시켰다가 필요할 때 다시 불러옴
- OOM(Out of Memory) 에러 없이 큰 모델 학습을 안정적으로 지속 가능

QLoRA Hyper-parameters

Quantization Hyper-parameter

Hyper-parameter	의미	일반적인 설정
load_in_4bit	Base Model을 4-bit로 로드	True
bnb_4bit_quant_type	4-bit 양자화 데이터 타입	"nf4"
bnb_4bit_use_double_quant	Quantization Constant도 다시 양자화	True
bnb_4bit_compute_dtype	실제 행렬 연산에 사용할 정밀도	torch.bfloat16

- ✓ QLoRA 설정은 3개 부분으로 구성 = Quantization + LoRA + Training
- ✓ bnb_4bit_compute_dtype : torch.bfloat16 (A100, H100 등 최신 GPU), torch.float16 (T4, V100 등 구형 GPU)
- ✓ bnb_4bit_use_double_quant : 성능 저하 거의 없고, 메모리는 추가 절감되어 관례적으로 True

QLoRA Hyper-parameters

Quantization Hyper-parameter

```
import torch
from transformers import BitsAndBytesConfig

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
)
```

sLLM 구현 및 Fine Tuning

Fine-Tuning Troubleshooting

- Common Problems, Causes, and Solutions
- Fine-Tuning Checklist

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Data Problems

모델은 데이터의 의도뿐 아니라 데이터에 포함된 오류와 편향까지 학습한다

Poor Dataset Quality

- 잘못된 정답, 중복 데이터, 불완전하거나 모순된 응답
- 샘플마다 다른 답변 형식·문체·상세 수준
- 소량의 고품질 데이터보다 대량 확보에 치중
- 데이터의 오류와 노이즈가 올바른 응답 패턴처럼 학습됨
- 결과적으로 응답 품질이 불안정해지고 잘못된 형식이 반복됨

Dataset Bias

- 특정 주제·표현·클래스·답변 유형에 편중된 데이터
- 유사한 예시만 반복되고 다양한 상황이나 예외 사례가 부족
- 실제 사용 환경과 학습 데이터 사이에 분포 차이 발생
- 자주 등장한 패턴을 과도하게 선호하고 드문 요청에는 취약
- 결과적으로 편향되거나 획일적인 응답을 생성

"Garbage In, Garbage Out"

"Biased Data, Biased Behavior"

How to Reduce the Risk

- Data Cleaning · Deduplication · Balanced Sampling
- Format Standardization · Quality Review

Training Problems

좋은 학습은 Train Loss를 낮추는 것이 아니라, 새로운 데이터에서도 성능을 높이는 것

Overfitting

- 학습 데이터에는 잘 답하지만 새로운 입력에서는 성능 저하
- 작거나 다양성이 부족한 데이터셋을 과도하게 반복 학습
- 학습 문장과 응답 형식을 일반화하지 못하고 그대로 암기
- Train Loss는 계속 감소하지만 Validation Loss는 증가
- 표현이 조금만 바뀌어도 응답 품질이 크게 저하됨

"Learning the Dataset, Not the Task"

Training Instability

- Loss가 크게 진동하거나 감소하지 않고 발산
- 지나치게 높은 Learning Rate로 가중치가 급격하게 변화
- 부적절한 Batch Size 또는 Gradient Accumulation 설정
- Gradient Explosion이나 수치 정밀도 문제 발생
- 실행할 때마다 학습 결과와 최종 성능이 크게 달라짐

"Unstable Loss and Gradients"

How to Reduce the Risk

- Overfitting: Fewer Epochs · Early Stopping · More Diverse Data
- Training Instability: Lower Learning Rate · Adjust Batch Size

Model Behavior Problems

모델은 원하는 답변 방식은 학습하면서도, 정확한 사실까지 학습하는 것은 아니다

Catastrophic Forgetting

- 새로운 능력을 학습하는 과정에서 기존 능력이 저하되는 현상
- 일반 지식, 추론, 언어 이해, 지시 수행 능력이 약화될 수 있음
- 특정 도메인이나 응답 형식에 편중된 데이터가 주요 원인
- 과도한 학습으로 기존 가중치가 크게 변경
- 특정 Task의 성능은 높아지지만 범용 성능은 오히려 하락

"New Skills, Lost Capabilities"

Hallucination

- Fine-Tuning이 사실성과 정확성을 자동으로 높이지는 않음
- 잘못되거나 불완전한 학습 데이터가 부정확한 답변을 강화
- 답을 알 수 없는 상황에서도 학습된 응답 형식을 모방
- 도메인 밖 질문에 대해서도 그럴듯한 답변 패턴을 반복
- 문체와 형식은 개선되지만 사실 오류와 과도한 확신은 남을 수 있음

"Fluent but Incorrect Answers"

How to Reduce the Risk

- Forgetting : Lower LR · Fewer Epochs · Mix General Data · PEFT
- Hallucination : Quality Data · Abstention Examples · RAG · Verification

Fine-Tuning Checklist

Improve target behavior without losing general capabilities

Before Training

- Fine-Tuning으로 해결할 문제와 목표 행동이 명확한가?
- 데이터의 정답·형식·문체가 정확하고 일관적인가?
- 중복 데이터와 지나치게 유사한 샘플을 제거했는가?
- 실제 사용 환경의 입력과 예외 상황을 포함하고 있는가?
- Train·Validation 데이터를 분리했는가?
- 학습 전 Base Model의 응답을 저장했는가?

During Training

- Train Loss와 Validation Loss를 함께 확인했는가?
- Validation Loss가 증가하기 시작하지 않는가?
- Learning Rate와 Epoch가 과도하지 않은가?
- Loss가 크게 진동하거나 NaN이 발생하지 않는가?
- 정기적으로 Checkpoint를 저장하고 있는가?
- 가장 좋은 Validation 성능의 Checkpoint를 선택했는가?

Prepare Data and Baseline

- Inspect Samples
- Remove Duplicates · Split Dataset
- Save Baseline Outputs

Monitor the Training Process

- Train/Validation Loss
- Learning Curve
- Saved Checkpoints
- Sample Outputs

Fine-Tuning Checklist

Improve target behavior without losing general capabilities

After Training

- 동일한 평가 질문에서 Base Model보다 목표 성능이 개선되었는가?
- 학습에 사용하지 않은 새로운 표현에도 잘 대응하는가?
- 학습 문장을 그대로 암기하거나 반복하지 않는가?
- 기존의 일반 지식·추론·지시 수행 능력이 유지되는가?
- 근거가 없을 때 추측하지 않고 답변을 유보하는가?
- 사실성·편향·환각을 별도의 평가 질문으로 확인했는가?

Evaluate Improvement

- Base vs. Fine-Tuned
- Seen vs. Unseen
- In-domain vs. Out-of-domain

평가 Prompt 유형

평가 유형	확인 목적
Target Task	학습하려던 행동이 개선되었는가?
Unseen Variation	표현이 달라져도 일반화되는가?
General / Out-of-domain	기존 능력 저하나 억지 답변이 없는가?

End of Documents

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.